

机可以在轻微拥塞时将经过的 RM 信元中的 NI 比特置为 1，在严重拥塞时，把 CI 比特置为 1。当目的主机收到一个 RM 信元时，它将把该 RM 信元发回给发送方，而保持 CI 与 NI 比特不变（除了 CI 比特也许会因为上面描述的 EFCI 机制而由目的端置为 1 之外）。

- ER 的设置。每一个 RM 信元还包含一个两字节的显式速率（Explicit Rate, ER）字段。一个拥塞的交换机也许会降低经过的 RM 信元中 ER 字段所包含的值。以这种方式，ER 字段将被设置为在源至目的地的路径上的所有交换机中的最小可支持速率。

一个 ATM ABR 源以返回的 RM 信元中的 CI、NI 及 ER 值为函数，来调整其发送信元的速率。进行速率调整的规则非常复杂而且繁琐，感兴趣的读者可以参考 [Jain 1996] 以得到详细信息。

3.7 TCP 拥塞控制

在本节中，我们再次来学习 TCP。如我们在 3.5 节所见，TCP 为运行在不同主机上的两个进程之间提供了可靠传输服务。TCP 的另一个关键部分就是其拥塞控制机制。如在前一节所指出，TCP 必须使用端到端拥塞控制而不是使网络辅助的拥塞控制，因为 IP 层不向端系统提供显式的网络拥塞反馈。

TCP 所采用的方法是让每一个发送方根据所感知到的网络拥塞程度来限制其能向连接发送流量的速率。如果一个 TCP 发送方感知从它到目的地之间的路径上没什么拥塞，则 TCP 发送方增加其发送速率；如果发送方感知沿着该路径有拥塞，则发送方就会降低其发送速率。但是这种方法提出了三个问题。第一，一个 TCP 发送方如何限制它向其连接发送流量的速率呢？第二，一个 TCP 发送方如何感知从它到目的地之间的路径上存在拥塞呢？第三，当发送方感知到端到端的拥塞时，采用何种算法来改变其发送速率呢？

我们首先分析一下 TCP 发送方是如何限制向其连接发送流量的。在 3.5 节中我们看到，TCP 连接的每一端都是由一个接收缓存、一个发送缓存和几个变量（LastByteRead、rwnd 等）组成。运行在发送方的 TCP 拥塞控制机制跟踪一个额外的变量，即拥塞窗口（congestion window）。拥塞窗口表示为 cwnd，它对一个 TCP 发送方能向网络中发送流量的速率进行了限制。特别是，在一个发送方中未被确认的数据量不会超过 cwnd 与 rwnd 中的最小值，即

$$\text{LastByteSent} - \text{LastByteAcked} \leq \min \{ \text{cwnd}, \text{rwnd} \}$$

为了关注拥塞控制（与流量控制形成对比），我们后面假设 TCP 接收缓存足够大，以至可以忽略接收窗口的限制；因此在发送方中未被确认的数据量仅受限于 cwnd。我们还假设发送方总是有数据要发送，即在拥塞窗口中的所有报文段要被发送。

上面的约束限制了发送方中未被确认的数据量，因此间接地限制了发送方的发送速率。为了理解这一点，我们来考虑一个丢包和发送时延均可以忽略不计的连接。因此粗略地讲，在每个往返时间（RTT）的起始点，上面的限制条件允许发送方向该连接发送 cwnd 个字节的数据，在该 RTT 结束时发送方接收对数据的确认报文。因此，该发送方的发送速率大概是 cwnd/RTT 字节/秒。通过调节 cwnd 的值，发送方因此能调整它向连接发送数据的速率。

我们接下来考虑 TCP 发送方是如何感知在它与目的地之间的路径上出现了拥塞的。我

们将一个 TCP 发送方的“丢包事件”定义为：要么出现超时，要么收到来自接收方的 3 个冗余 ACK。（回想我们在 3.5.4 节有关图 3-33 中的超时事件的讨论和收到 3 个冗余 ACK 后包括快速重传的后继修改。）当出现过度的拥塞时，在沿着这条路径上的一台（或多台）路由器的缓存会溢出，引起一个数据报（包含一个 TCP 报文段）被丢弃。丢弃的数据报接着会引起发送方的丢包事件（要么超时或收到 3 个冗余 ACK），发送方就认为在发送方到接收方的路径上出现了拥塞的指示。

考虑了拥塞检测问题后，我们接下来考虑网络没有拥塞这种更为乐观的情况，即没有出现丢包事件的情况。在此情况下，在 TCP 的发送方将收到对于以前未确认报文段的确认。如我们将看到的那样，TCP 将这些确认的到达作为一切正常的指示，即在网络上传输的报文段正被成功地交付给目的地，并使用确认来增加窗口的长度（及其传输速率）。注意到如果确认以相当慢的速率到达（例如，如果该端到端路径具有高时延或包含一段低带宽链路），则该拥塞窗口将以相当慢的速率增加。在另一方面，如果确认以高速率到达，则该拥塞窗口将会更为迅速地增大。因为 TCP 使用确认来触发（或计时）增大它的拥塞窗口长度，TCP 被说成是自计时（self-clocking）的。

给定调节 cwnd 值以控制发送速率的机制，关键的问题依然存在：TCP 发送方怎样确定它应当发送的速率呢？如果众多 TCP 发送方总体上发送太快，它们能够拥塞网络，导致我们在图 3-48 中看到的拥塞崩溃。事实上，为了应对在较早 TCP 版本下观察到的因特网拥塞崩溃 [Jacobson 1988]，研发了该版本的 TCP（我们马上将学习它）。然而，如果 TCP 发送方过于谨慎，发送太慢，它们不能充分利用网络的带宽；这就是说，TCP 发送方能够以更高的速率发送而不会使网络拥塞。那么 TCP 发送方如何确定它们的发送速率，既使得网络不会拥塞，与此同时又能充分利用所有可用的带宽？TCP 发送方是显式地协作，或存在一种分布式方法使 TCP 发送方能够仅基于本地信息设置它们的发送速率？TCP 使用下列指导性原则回答这些问题：

- 一个丢失的报文段表意味着拥塞，因此当丢失报文段时应当降低 TCP 发送方的速率。回想在 3.5.4 节中的讨论，对于给定报文段，一个超时事件或四个确认（一个初始 ACK 和其后的三个冗余 ACK）被解释为跟随该四个 ACK 的报文段的“丢包事件”的一种隐含的指示。从拥塞控制的观点看，该问题是 TCP 发送方应当如何减小它的拥塞窗口长度，即减小其发送速率，以应对这种推测的丢包事件。
- 一个确认报文段指示该网络正在向接收方交付发送方的报文段，因此，当对先前未确认报文段的确认到达时，能够增加发送方的速率。确认的到达被认为是一切顺利的隐含指示，即报文段正从发送方成功地交付给接收方，因此该网络不拥塞。拥塞窗口长度因此能够增加。
- 带宽探测。给定 ACK 指示源到目的地路径无拥塞，而丢包事件指示路径拥塞，TCP 调节其传输速率的策略是增加其速率以响应到达的 ACK，除非出现丢包事件，此时才减小传输速率。因此，为探测拥塞开始出现的速率，TCP 发送方增加它的传输速率，从该速率后退，进而再次开始探测，看看拥塞开始速率是否发生了变化。TCP 发送方的行为也许类似于要求（并得到）越来越多糖果的孩子，直到最后告知他/她“不行！”，孩子后退一点，然后过一会儿再次开始提出请求。注意到网络中没有明确的拥塞状态信令，即 ACK 和丢包事件充当了隐式信号，并且每个 TCP 发送方根据异步于其他 TCP 发送方的本地信息而行动。

概述了 TCP 拥塞控制后，现在是我们考虑广受赞誉的 TCP 拥塞控制算法（TCP congestion control algorithm）细节的时候了，该算法首先在 [Jacobson 1988] 中描述并且在 [RFC 5681] 中标准化。该算法包括 3 个主要部分：①慢启动；②拥塞避免；③快速恢复。慢启动和拥塞避免是 TCP 的强制部分，两者的差异在于对收到的 ACK 做出反应时增加 cwnd 长度的方式。我们很快将会看到慢启动比拥塞避免能更快地增加 cwnd 的长度（不要被名称所迷惑！）。快速恢复是推荐部分，对 TCP 发送方并非是必需的。

1. 慢启动

当一条 TCP 连接开始时，cwnd 的值通常初始置为一个 MSS 的较小值 [RFC 3390]，这就使得初始发送速率大约为 MSS/RTT 。例如，如果 $MSS = 500$ 字节且 $RTT = 200ms$ ，则得到的初始发送速率大约只有 20kbps。由于对 TCP 发送方而言，可用带宽可能比 MSS/RTT 大得多，TCP 发送方希望迅速找到可用带宽的数量。因此，在慢启动（slow-start）状态，cwnd 的值以 1 个 MSS 开始并且每当传输的报文段首次被确认就增加 1 个 MSS。在图 3-51 所示的例子中，TCP 向网络发送第一个报文段并等待一个确认。当该确认到达时，TCP 发送方将拥塞窗口增加一个 MSS，并发送出两个最大长度的报文段。这两个报文段被确认，则发送方对每个确认报文段将拥塞窗口增加一个 MSS，使得拥塞窗口变为 4 个 MSS，并这样下去。这一过程每过一个 RTT，发送速率就翻番。因此，TCP 发送速率起始慢，但在慢启动阶段以指数增长。

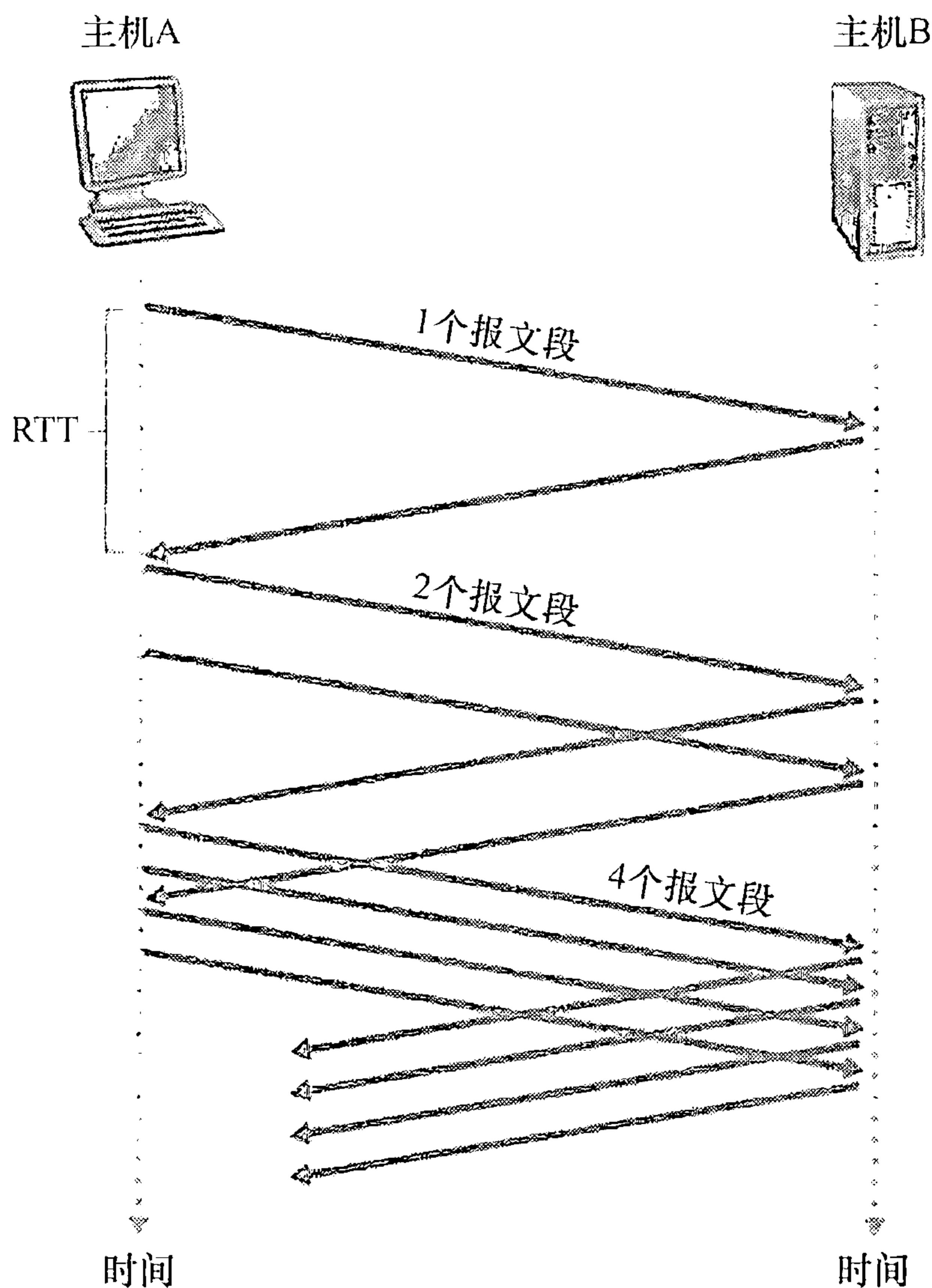


图 3-51 TCP 慢启动

但是，何时结束这种指数增长呢？慢启动对这个问题提供了几种答案。首先，如果存在一个由超时指示的丢包事件（即拥塞），TCP 发送方将 cwnd 设置为 1 并重新开始慢启动过程。它还将第二个状态变量的值 ssthresh（“慢启动阈值”的速记）设置为 $cwnd/2$ ，即当检测到拥塞时将 ssthresh 置为拥塞窗口值的一半。慢启动结束的第二种方式是直接与 ssthresh 的值相关联。因为当检测到拥塞时 ssthresh 设为 cwnd 的值一半，当到达或超过 ssthresh 的值时，继续使 cwnd 翻番可能有些鲁莽。因此，当 cwnd 的值等于 ssthresh 时，结束慢启动并且 TCP 转移到拥塞避免模式。我们将会看到，当进入拥塞避免模式时，TCP 更为谨慎地增加 cwnd。最后一种结束慢启动的方式是，如果检测到 3 个冗余 ACK，这时 TCP 执行一种快速重传（参见 3.5.4 节）并进入快速恢复状态，后面将讨论相关内容。慢启动中的 TCP 行为总结在图 3-52 中的 TCP 拥塞控制的 FSM 描述中。慢启动算法最早源于 [Jacobson 1988]；在 [Jain 1986] 中独立地提出了一种类似于慢启动的方法。

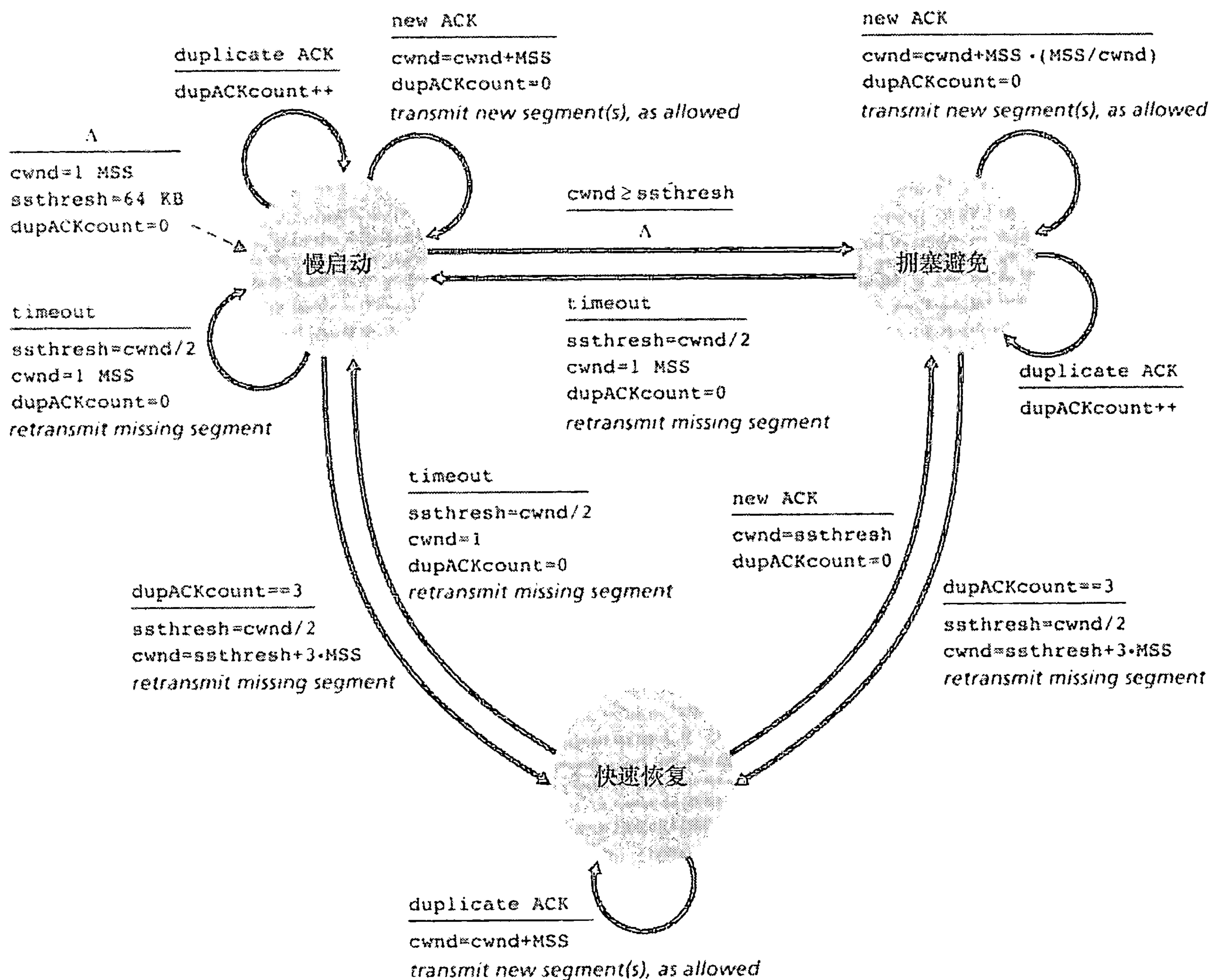


图 3-52 TCP 拥塞控制的 FSM 描述

实践原则

TCP 分岔：优化云服务的性能

对于诸如搜索、电子邮件和社交网络等云服务，非常希望提供高水平的响应性，给用户一种完美的印象，即这些服务正运行在它们自己的端系统（包括其智能手机）中。因为用户经常位于远离数据中心的地方，而这些数据中心负责为云服务关联的动态内容提供服务。实际上，如果端系统远离数据中心，则 RTT 将会很大，会由于 TCP 慢启动潜在地导致低劣的响应时间性能。

作为一个学习案例，考虑接收对某搜索问题的响应中的时延。通常，服务器在慢启动期间交付响应要求三个 TCP 窗口 [Pathak 2010]。所以从某端系统发起一条 TCP 连接到它收到该响应的最后一个分组的时间粗略是 $4 * RTT$ （用于建立 TCP 连接的一个 RTT 加上用于 3 个数据窗口的 3 个 RTT），再加上在数据中心的处理时间。对于一个相当小的查询来说，这些 RTT 时延导致其返回搜索结果中显而易见的时延。此外，在接入网中可能有较大的丢包，导致 TCP 重传甚至较大的时延。

缓解这个问题和改善用户感受到的性能的一个途径是：①部署邻近用户的前端服务器；②在该前端服务器利用 TCP 分岔（TCP splitting）来分裂 TCP 连接。借助于 TCP 分

岔，客户向邻近前端连接一条 TCP 连接，并且该前端以非常大的窗口向数据中心维护一条 TCP 连接 [Tariq 2008, Pathak 2010, Chen 2011]。使用这种方法，响应时间大致变为 $4 * RTT_{FE} + RTT_{BE} + \text{处理时间}$ ，其中 RTT_{FE} 是客户与前端服务器之间的往返时间， RTT_{BE} 是前端服务器与数据中心（后端服务器）之间的往返时间。如果前端服务器邻近客户，则该响应时间大约变为 RTT_{BE} 加上处理时间，因为 RTT_{FE} 小得微不足道并且 RTT_{BE} 约为 RTT 。总而言之，TCP 分岔大约能够将网络时延从 $4 * RTT$ 减少到 RTT ，极大地改善用户感受的性能，对于远离最近数据中心的用户更是如此。TCP 分岔也有助于减少因接入网丢包引起的 TCP 重传时延。今天，Google 和 Akamai 在接入网中广泛利用了它们的 CDN 服务器（参见 7.2 节），为它们支持的云服务来执行 TCP 分岔 [Chen 2011]。

2. 拥塞避免

一旦进入拥塞避免状态， $cwnd$ 的值大约是上次遇到拥塞时的值的一半，即距离拥塞可能并不遥远！因此，TCP 无法每过一个 RTT 再将 $cwnd$ 的值翻番，而是采用了一种较为保守的方法，每个 RTT 只将 $cwnd$ 的值增加一个 MSS [RFC 5681]。这能够以几种方式完成。一种通用的方法是对于 TCP 发送方无论何时到达一个新的确认，就将 $cwnd$ 增加一个 MSS ($MSS/cwnd$) 字节。例如，如果 MSS 是 1460 字节并且 $cwnd$ 是 14 600 字节，则在一个 RTT 内发送 10 个报文段。每个到达 ACK（假定每个报文段一个 ACK）增加 $1/10MSS$ 的拥塞窗口长度，因此在收到对所有 10 个报文段的确认后，拥塞窗口的值将增加了一个 MSS 。

但是何时应当结束拥塞避免的线性增长（每 RTT 1MSS）呢？当出现超时，TCP 的拥塞避免算法行为相同。与慢启动的情况一样， $cwnd$ 的值被设置为 1 个 MSS ，当丢包事件出现时， $ssthresh$ 的值被更新为 $cwnd$ 值的一半。然而，前面讲过丢包事件也能由一个三个冗余 ACK 事件触发。在这种情况下，网络继续从发送方向接收方交付报文段（就像由收到冗余 ACK 所指示的那样）。因此 TCP 对这种丢包事件的行为，相比于超时指示的丢包，应当不那么剧烈：TCP 将 $cwnd$ 的值减半（为使测量结果更好，计及已收到的 3 个冗余的 ACK 要加上 3 个 MSS ），并且当收到 3 个冗余的 ACK，将 $ssthresh$ 的值记录为 $cwnd$ 的值的一半。接下来进入快速恢复状态。

3. 快速恢复

在快速恢复中，对于引起 TCP 进入快速恢复状态的缺失报文段，对收到的每个冗余的 ACK， $cwnd$ 的值增加一个 MSS 。最终，当对丢失报文段的一个 ACK 到达时，TCP 在降低 $cwnd$ 后进入拥塞避免状态。如果出现超时事件，快速恢复在执行如同在慢启动和拥塞避免中相同的动作后，迁移到慢启动状态：当丢包事件出现时， $cwnd$ 的值被设置为 1 个 MSS ，并且 $ssthresh$ 的值设置为 $cwnd$ 值的一半。

快速恢复是 TCP 推荐的而非必需的构件 [RFC 5681]。有趣的是，一种称为 TCP Tahoe 的 TCP 早期版本，不管是发生超时指示的丢包事件，还是发生 3 个冗余 ACK 指示的丢包事件，都无条件地将其拥塞窗口减至 1 个 MSS ，并进入慢启动阶段。TCP 的较新版本 TCP Reno，则综合了快速恢复。

图 3-53 图示了 Reno 版 TCP 与 Tahoe 版 TCP 的拥塞控制窗口的演化情况。在该图中，

阈值初始等于 8 个 MSS。在前 8 个传输回合，Tahoe 和 Reno 采取了相同的动作。拥塞窗口在慢启动阶段以指数速度快速爬升，在第 4 轮传输时到达了阈值。然后拥塞窗口以线性速度爬升，直到在第 8 轮传输后出现 3 个冗余 ACK。注意到当该丢包事件发生时，拥塞窗口值为 $12 \times \text{MSS}$ 。于是 ssthresh 的值被设置为 $0.5 \times \text{cwnd} = 6 \times \text{MSS}$ 。在 TCP Reno 下，拥塞窗口被设置为 $\text{cwnd} = 9\text{MSS}$ ，然后线性地增长。在 TCP Tahoe 下，拥塞窗口被设置为 1 个 MSS，然后呈指数增长，直至到达 ssthresh 值为止，在这个点它开始线性增长。

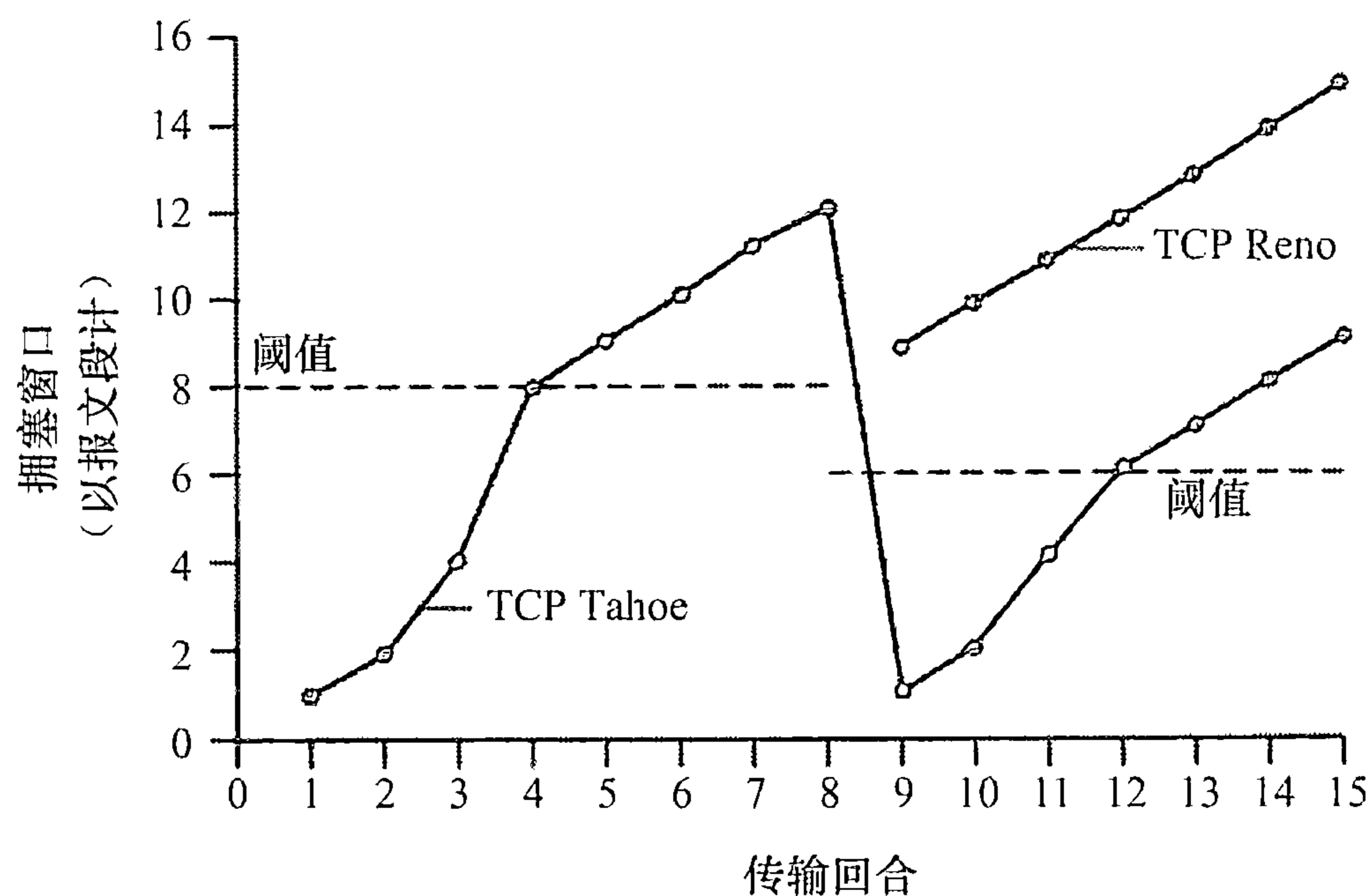


图 3-53 TCP 拥塞窗口的演化 (Tahoe 和 Reno)

图 3-52 表示了 TCP 拥塞控制算法（即慢启动、拥塞避免和快速恢复）的完整 FSM 描述。该图也指示了新报文段的传输或重传的报文段可能出现的位置。尽管区分 TCP 差错控制/重传与 TCP 拥塞控制非常重要，但是注意到 TCP 这两个方面交织链接的方式也很重要。

4. TCP 拥塞控制：回顾

在深入了解慢启动、拥塞避免和快速恢复的细节后，现在有必要退回来回顾一下全局。忽略一条连接开始时初始的慢启动阶段，假定丢包由 3 个冗余的 ACK 而不是超时指示，TCP 的拥塞控制是：每个 RTT 内 cwnd 线性（加性）增加 1MSS，然后出现 3 个冗余 ACK 事件时 cwnd 减半（乘性减）。因此，TCP 拥塞控制常常被称为加性增、乘性减（Additive-Increase, Multiplicative-Decrease, AIMD）拥塞控制方式。AIMD 拥塞控制引发了在图 3-54 中所示的“锯齿”行为，这也很好地图示了我们前面 TCP 检测带宽时的直觉，即 TCP 线性地增加它的拥塞窗口长度（因此增加其传输速率），直到出现 3 个冗余 ACK 事件。然后以 2 个因子来减少它的拥塞窗口长度，然后又开始了线性增长，探测是否还有另外的可用带宽。

如前所述，许多 TCP 实现采用了 Reno 算法 [Padhye 2001]。Reno 算法的许多变种已被提出 [RFC 3782; RFC 2018]。TCP Vegas 算法 [Brakmo 1995; Ahn 1995] 试图在维持较好的吞吐量的同时避免拥塞。Vegas 的基本思想是：①在分组丢失发生之前，在源与目的地之间检测路由器中的拥塞；②当检测出快要发生的分组丢失时，线性地降低发送速率。快要发生的分组丢失是通过观察 RTT 来预测的。分组的 RTT 越长，路由器中的拥塞越严重。Linux 支持若干拥塞控制算法（包括 TCP Reno 和 TCP Vegas），并且允许系统管理员配置使用哪个版本的 TCP。在 Linux 版本 2.6.18 中，TCP 默认版本设置为 CUBIC [Ha 2008]，这是为高带宽应用开发的一个 TCP 版本。对于许多特色 TCP 的综述参见 [Afanasyev 2010]。

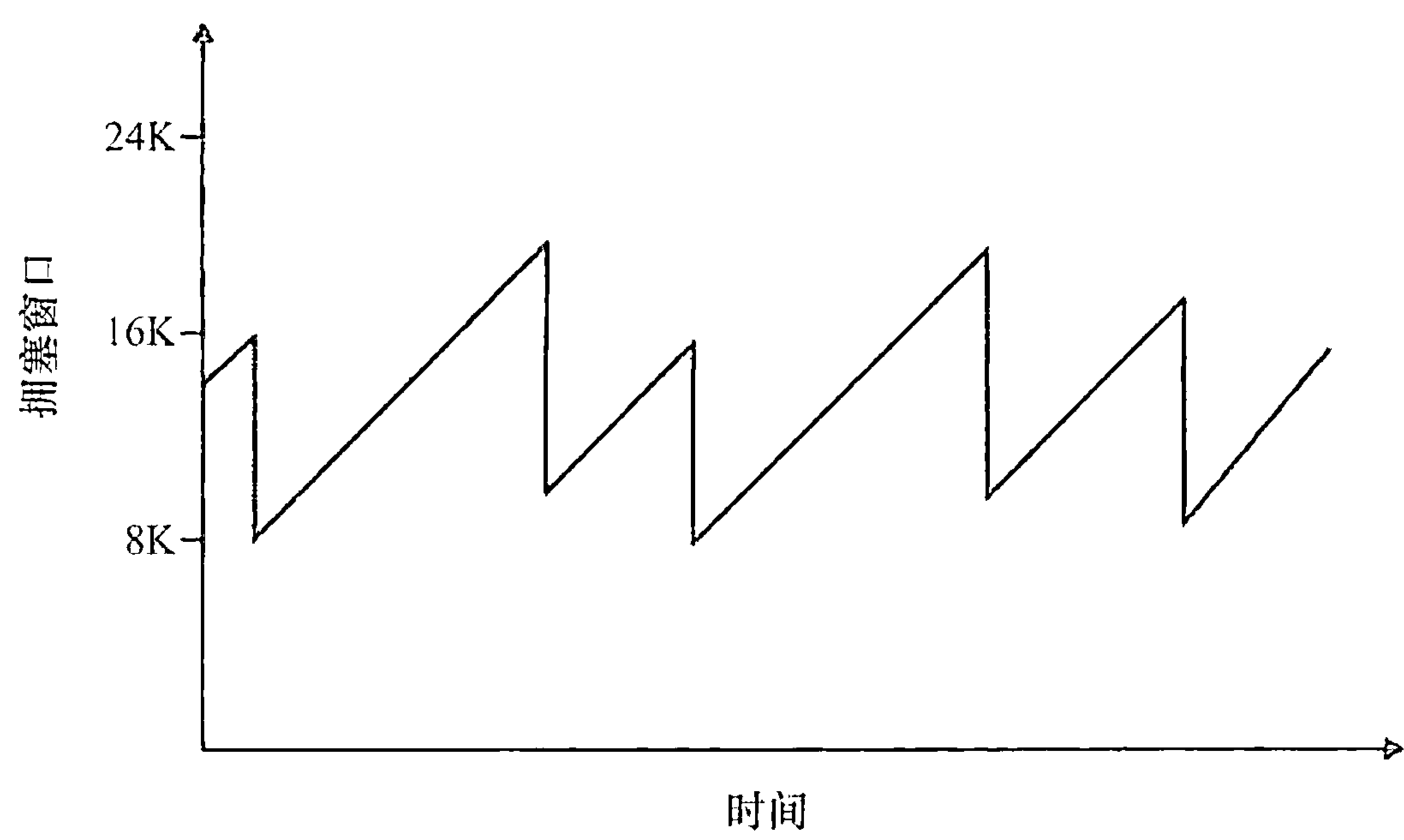


图 3-54 加性增、乘性减的拥塞控制

TCP AIMD 算法基于大量的工程见解和在运行网络中的拥塞控制经验而开发。在 TCP 研发后的十年，理论分析显示 TCP 的拥塞控制算法用做一种分布式异步优化算法，使得用户和网络性能的几个重要方面被同时优化 [Kelly 1998]。拥塞控制的丰富理论已经得到发展 [Srikant 2004]。

5. 对 TCP 吞吐量的宏观描述

给出 TCP 的锯齿状行为后，自然要考虑一个长存活期的 TCP 连接的平均吞吐量（即平均速率）可能是多少。在这个分析中，我们将忽略在超时事件后出现的慢启动阶段。（这些阶段通常非常短，因为发送方很快就以指数增长离开该阶段。）在一个特定的往返间隔内，TCP 发送数据的速率是拥塞窗口与当前 RTT 的函数。当窗口长度是 w 字节，且当前往返时间是 RTT 秒时，则 TCP 的发送速率大约是 w/RTT 。于是，TCP 通过每经过 1 个 RTT 将 w 增加 1 个 MSS 探测出额外的带宽，直到一个丢包事件发生为止。当一个丢包事件发生时，用 W 表示 w 的值。假设在连接持续期间 RTT 和 W 几乎不变，那么 TCP 的传输速率在 $W/(2 \times RTT)$ 到 W/RTT 之间变化。

这些假设导出了 TCP 稳态行为的一个高度简化的宏观模型。当速率增长至 W/RTT 时，网络丢弃来自连接的分组；然后发送速率就会减半，进而每过一个 RTT 就发送速率增加 MSS/RTT ，直到再次达到 W/RTT 为止。这一过程不断地自我重复。因为 TCP 吞吐量（即速率）在两个极值之间线性增长，所以我们有

$$\text{一条连接的平均吞吐量} = \frac{0.75 \times W}{RTT}$$

通过这个高度理想化的 TCP 稳态动态性模型，我们可以推出一个将连接的丢包率与可用带宽联系起来的有趣表达式 [Mahdavi 1997]。这个推导将在课后习题中概要给出。一个根据经验建立的并与测量数据一致的更复杂模型参见 [Padhye 2000]。

6. 经高带宽路径的 TCP

认识到下列事实是重要的：TCP 拥塞控制已经演化了多年并仍在继续演化。对当前 TCP 变量的总结和 TCP 演化的讨论，参见 [Floyd 2001, RFC 5681, Afanasyev 2010]。以往对因特网有益的东西（那时大量的 TCP 连接承载的是 SMTP、FTP 和 Telnet 流量），不一定对当今 HTTP 主宰的因特网或具有难以想象的服务的未来因特网还是有益的。

TCP 继续演化的需求能够通过考虑网格和云计算应用所需要的高速 TCP 连接加以阐述。例如，考虑一条具有 1500 字节报文段和 100ms RTT 的 TCP 连接，假定我们要通过这条连接以 10Gbps 速率发送数据。根据 [RFC 3649]，我们注意到使用上述 TCP 吞吐量公式，为了取得 10Gbps 吞吐量，平均拥塞窗口长度将需要是 83 333 个报文段。对如此大量的报文段，使我们相当关注这 83 333 个传输中的报文段也许会丢失。在丢失的情况下，将会出现什么情况呢？或者以另一种方式说，这些传输的报文段能以何种比例丢失，使得在图 3-52 中列出的 TCP 拥塞控制算法仍能取得所希望的 10Gbps 速率？在本章的课后习题中，要求读者推导出一条 TCP 连接的吞吐量公式，该公式作为丢包率 (L)、往返时间 (RTT) 和最大报文段长度 (MSS) 的函数：

$$\text{一条连接的平均吞吐量} = \frac{1.22 \times \text{MSS}}{\text{RTT} \sqrt{L}}$$

使用该公式，我们能够看到，为了取得 10Gbps 的吞吐量，今天的 TCP 拥塞控制算法仅能容忍 2×10^{-10} 的报文段丢失概率（或等价地说，对每 5 000 000 000 个报文段有一个丢包），这是一个非常低的值。这种观察导致许多研究人员为这种高速环境特别设计新版 TCP，对这些努力的讨论参见 [Jin 2004, RFC 3649, Kelly 2003, Ha 2008]。

公平性

考虑 K 条 TCP 连接，每条都有不同的端到端路径，但是都经过一段传输速率为 R bps 的瓶颈链路。（所谓瓶颈链路，是指对于每条连接，沿着该连接路径上的所有其他段链路都不拥塞，而且与该瓶颈链路的传输容量相比，它们都有充足的传输容量。）假设每条连接都在传输一个大文件，而且无 UDP 流量通过该段瓶颈链路。如果每条连接的平均传输速率接近 R/K ，即每条连接都得到相同份额的链路带宽，则认为该拥塞控制机制是公平的。

TCP 的 AIMD 算法公平吗？尤其是假定可在不同时间启动并因此在某个给定的时间点可能具有不同的窗口长度情况下，对这些不同的 TCP 连接还是公平的吗？TCP 趋于在竞争的多条 TCP 连接之间提供对一段瓶颈链路带宽的平等分享，其理由 [Chiu 1989] 给出了一个极好的、直观的解释。

我们考虑有两条 TCP 连接共享一段传输速率为 R 的链路的简单例子，如图 3-55 中所示。我们将假设这两条连接有相同的 MSS 和 RTT（这样如果它们有相同的拥塞窗口长度，就会有相同的吞吐量），它们有大量的数据要发送，且没有其他 TCP 连接或 UDP 数据报穿越该段共享链路。我们还将忽略 TCP 的慢启动阶段，并假设 TCP 连接一直按 CA 模式 (AIMD) 运行。

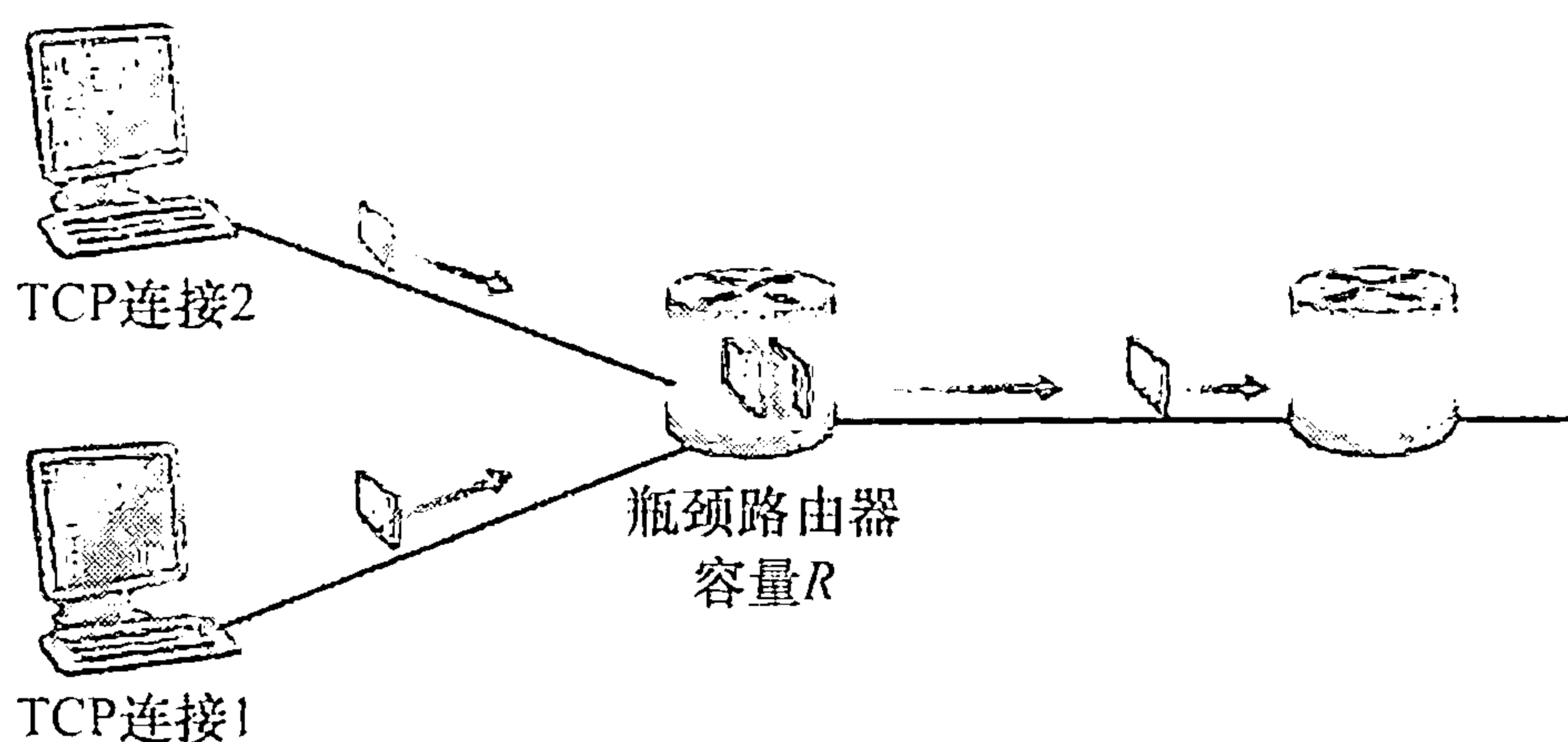


图 3-55 两条 TCP 连接共享同一条瓶颈链路

图 3-56 描绘了两条 TCP 连接实现的吞吐量情况。如果 TCP 要在这两条 TCP 连接之间平等地共享链路带宽，那么实现的吞吐量曲线应当是从原点沿 45° 方向的箭头向外辐射（平等带宽共享）。理想情况是，两个吞吐量的和应等于 R 。（当然，每条连接得到相同但容量为 0 的共享链路容量并非我们所期望的情况！）所以我们的目标应该是使取得的吞吐量落在图 3-56 中平等带宽共享曲线与全带宽利用曲线的交叉点附近的某处。

假定 TCP 窗口长度是这样的，即在某给定时刻，连接 1 和连接 2 实现了由图 3-56 中 A 点所指明的吞吐量。因为这两条连接共同消耗的链路带宽量小于 R ，所以无丢包事件发生，根据 TCP 的拥塞避免算法的结果，这两条连接每过一个 RTT 都要将其窗口增加 1 个 MSS。因此，这两条连接的总吞吐量就会从 A 点开始沿 45° 线前行（两条连接都有相同的增长）。

最终，这两条连接共同消耗的带宽将超过 R ，最终将发生分组丢失。假设连接 1 和连接 2 实现 B 点指明的吞吐量时，它们都经历了分组丢失。连接 1 和连接 2 于是就按二分之一减小其窗口。所产生的结果实现了 C 点指明的吞吐量，它正好位于始于 B 点止于原点的一个向量的中间。因为在 C 点，共同消耗的带宽小于 R ，所以这两条连接再次沿着始于 C 点的 45° 线增加其吞吐量。最终，再次发生丢包事件，如在 D 点，这两条连接再次将其窗口长度减半，如此等等。你应当搞清楚这两条连接实现的带宽最终将沿着平等带宽共享曲线在波动。还应该搞清楚无论这两条连接位于二维空间的何处，它们最终都会收敛到该状态！虽然此时我们做了许多理想化的假设，但是它仍然能对解释为什么 TCP 会导致在多条连接之间的平等共享带宽这个问题提供一个直观的感觉。

在理想化情形中，我们假设仅有 TCP 连接穿过瓶颈链路，所有的连接具有相同的 RTT 值，且对于一个主机 - 目的地对而言只有一条 TCP 连接与之相关联。实践中，这些条件通常是得不到满足的，客户 - 服务器应用因此能获得非常不平等的链路带宽份额。特别是，已经表明当多条连接共享一个共同的瓶颈链路时，那些具有较小 RTT 的连接能够在链路空闲时更快地抢到可用带宽（即较快地打开其拥塞窗口），因而将比那些具有较大 RTT 的连接享用更高的吞吐量 [Laksman 1997]。

1. 公平性和 UDP

我们刚才已经看到，TCP 拥塞控制是如何通过拥塞窗口机制来调节一个应用程序的传输速率的。许多多媒体应用如因特网电话和视频会议，经常就因为这种特定原因而不在 TCP 上运行，因为它们不想其传输速率被扼制，即使在网络非常拥塞的情况下。相反，这些应用宁可在 UDP 上运行，UDP 是没有内置的拥塞控制的。当运行在 UDP 上时，这些应用能够以恒定的速率将其音频和视频数据注入网络之中并且偶尔会丢失分组，而不愿在拥塞时将其发送速率降至“公平”级别并且不丢失任何分组。从 TCP 的观点来看，运行在 UDP 上的多媒体应用是不公平的，因为它们不与其他连接合作，也不适时地调整其传输速

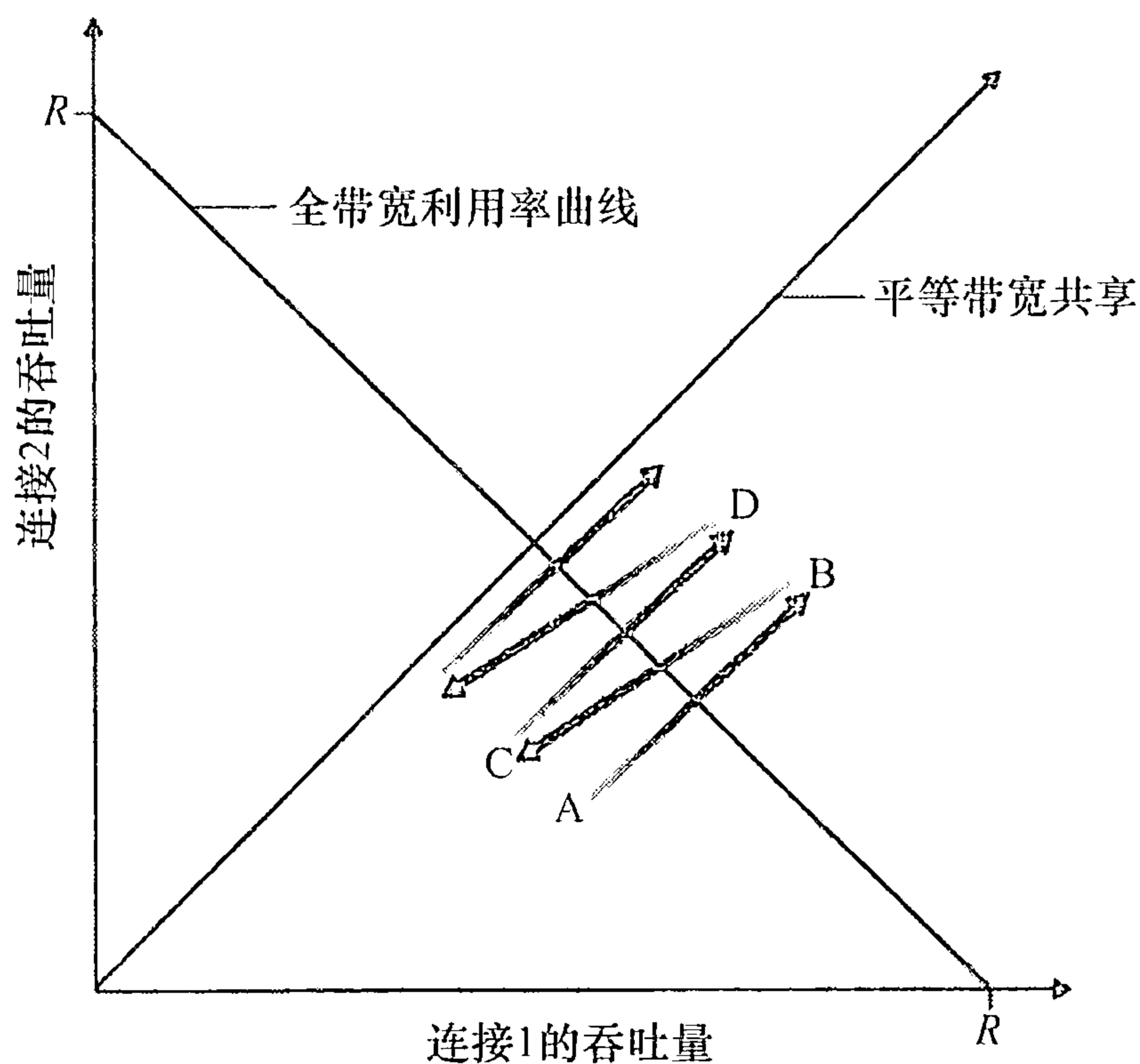


图 3-56 TCP 连接 1 和连接 2 实现的吞吐量

率。因为 TCP 拥塞控制在面临拥塞增加（丢包）时，将降低其传输速率，而 UDP 源则不必这样做，UDP 源有可能压制 TCP 流量。当今的一个主要研究领域就是开发一种因特网中的拥塞控制机制，用于阻止 UDP 流量不断压制直至中断因特网吞吐量的情况 [Floyd 1999; Floyd 2000; Kohler 2006]。

2. 公平性和并行 TCP 连接

即使我们能够迫使 UDP 流量具有公平的行为，但公平性问题仍然没有完全解决。这是因为我们没有什么办法阻止基于 TCP 的应用使用多个并行连接。例如，Web 浏览器通常使用多个并行 TCP 连接来传送一个 Web 页中的多个对象。（多条连接的确切数目可以在多数浏览器中进行配置。）当一个应用使用多条并行连接时，它占用了一条拥塞链路中较大比例的带宽。举例来说，考虑一段速率为 R 且支持 9 个在线客户 - 服务器应用的链路，每个应用使用一条 TCP 连接。如果一个新的应用加入进来，也使用一条 TCP 连接，则每个应用得到差不多相同的传输速率 $R/10$ 。但是如果这个新的应用这次使用了 11 个并行 TCP 连接，则这个新应用就不公平地分到超过 $R/2$ 的带宽。Web 流量在因特网中是非常普遍的，所以多条并行连接并非不常见。

3.8 小结

本章我们首先学习了运输层协议能够向网络应用程序提供的服务。在一个极端，运输层协议非常简单，并向应用程序不提供不必要的服务，而仅向通信进程提供多路复用/分解的功能。因特网中的 UDP 协议就是这样一种不提供不必要服务的运输层协议。在另一个极端，运输层协议能够向应用程序提供各种各样的保证，例如数据的可靠交付、时延保证和带宽保证。无论如何，运输层协议能够提供的服务经常受下面网络层协议服务模型的限制。如果网络层协议不能向运输层报文段提供时延或带宽保证，那么运输层协议就不能向进程间发送的报文提供时延或带宽保证。

在 3.4 节中，我们学习了运输层协议能够提供可靠数据传输，即使下面的网络层是不可靠的。我们看到了提供可靠的数据传送会遇到许多微妙的问题，但都可以通过精心地结合确认、定时器、重传以及序号机制来完成任务。

尽管在本章中我们包含了可靠数据传送，但是我们应该理解在链路层、网络层、运输层或应用层协议中都可以提供可靠数据传送。该协议栈中上面 4 层的任意一层都可以实现确认、定时器、重传以及序号，能够向其上层提供可靠数据传送。事实上，在过去数年中，工程师以及计算机科学家们已经独立地设计并实现了提供可靠数据传送的链路层、网络层、运输层以及应用层协议（虽然这些协议中的许多已经销声匿迹了）。

在 3.5 节中，我们详细地研究了 TCP 协议，它是因特网中面向连接和可靠的运输层协议。我们知道 TCP 是非常复杂的，它涉及了连接管理、流量控制、往返时间估计以及可靠数据传送。事实上，TCP 比我们描述的要更为复杂，即我们有意地避而不谈在各种 TCP 实现版本中广泛实现的各种 TCP 补丁、修复和改进。然而，所有这些复杂性都对网络层应用隐藏了起来。如果某主机上的客户希望向另一台主机上的服务器可靠地发送数据，它只需要打开对该服务器的一个 TCP 套接字，然后将数据注入该套接字。客户 - 服务器应用程序则乐于对 TCP 的复杂性视而不见。

在 3.6 节中，我们从广泛的角度研究了拥塞控制，在 3.7 节中我们阐述了 TCP 是如何

实现拥塞控制的。我们知道了拥塞控制对于网络良好运行是必不可少的。没有拥塞控制，网络很容易出现死锁，使得端到端之间很少或没有数据能被传输。在 3.7 节中我们学习了 TCP 实现的一种端到端拥塞控制机制，即当 TCP 连接的路径上判断不拥塞时，其传输速率就加性增；当出现丢包时，传输速率就乘性减。这种机制也致力于做到每一个通过拥塞链路的 TCP 连接能平等地共享该链路带宽。我们也深入探讨了 TCP 连接建立和慢启动对时延的影响。我们观察到在许多重要场合，连接建立和慢启动会对端到端时延产生严重影响。我们再次强调，尽管 TCP 在这几年一直在发展，但它仍然是一个值得深入研究的领域，并且在未来的几年中还可能持续演化。

在本章中我们对特定因特网运输协议的讨论集中在 UDP 和 TCP 上，它们是因特网运输层的两匹“驮马”。然而，对这两个协议的二十多年的经验已经使人们认识到，这两个协议都不是完美无缺的。研究人员因此在忙于研制其他的运输层协议，其中的几种现在已经成为 IETF 建议的标准。

数据报拥塞控制协议（Datagram Congestion Control Protocol, DCCP）[RFC 4340] 提供了一种低开销、面向报文、类似于 UDP 的不可靠服务，但是具有应用程序可选择的拥塞控制形式，该机制与 TCP 相兼容。如果某应用程序需要可靠的或半可靠的数据传送，则这将在应用程序自身中执行（也许使用我们已经在 3.4 节中学过的机制）。DCCP 被设想用于诸如流媒体（参见第 7 章）等应用程序中，DCCP 能够利用数据交付的预定时间和可靠性之间的折中，但是要对网络拥塞作出响应。

流控制传输协议（Stream Control Transmission Protocol, SCTP）[RFC 4960, RFC 3286] 是一种可靠的、面向报文的协议，该协议允许几个不同的应用层次的“流”复用到单个 SCTP 连接上（一种称之为“多流”的方法）。从可靠性的角度看，在该连接中的不同流被分别处理，因此在一条流中的分组丢失不会影响其他流中数据的交付。当一台主机与两个或更多个网络连接时，SCTP 也允许数据经两条出路径传输，还具有失序数据的选项交付和一些其他特色。SCTP 的流控制和拥塞控制算法基本上与 TCP 中的相同。

TCP 友好速率控制（TCP-Friendly Rate Control, TFRC）协议 [RFC 5348] 是一种拥塞控制协议而不是一种功能齐全的运输层协议。它定义了一种拥塞控制机制，该机制能被用于诸如 DCCP 等其他运输协议（事实上在 DCCP 中可供使用的两种应用程序可选的协议之一就是 TFRC）。TFRC 的目标是平滑在 TCP 拥塞控制中的“锯齿”行为（参见图 3-54），同时维护一种长期的发送速率，该速率“合理地”接近 TCP 的速率。使用比 TCP 更为平滑的发送速率，TFRC 非常适合诸如 IP 电话或流媒体等多媒体应用，这种平滑的速率对于这些应用是重要的。TFRC 是一种“基于方程”的协议，这些协议使用测得的丢包率作为方程的输入 [Padhye 2000]，即使用方程估计一个 TCP 会话在该丢包率下 TCP 的吞吐量将是多大。该速率则被取为 TFRC 的目标发送速率。

唯有未来才能告诉我们 DCCP、SCTP 或 TFRC 是否能得到广泛实施。虽然这些协议明确地提供了超过 TCP 和 UDP 的强化能力，但是多年来已经证明了 TCP 和 UDP 自身是“足够好”的。是否“更好”将胜出“足够好”，这将取决于技术、社会和商业考虑的复杂组合。

在第 1 章中，我们讲到计算机网络能被划分成“网络边缘”和“网络核心”。网络边缘包含了在端系统中发生的所有事情。既然已经覆盖了应用层和运输层，我们关于网络边缘的讨论也就结束了。接下来是探寻网络核心的时候了！我们的旅程从下一章开始，下一章将学习网络层，并且将在第 5 章继续学习链路层。

课后习题和问题



复习题

3.1 ~ 3.3 节

- R1. 假定网络层提供了下列服务。在源主机中的网络层接受最大长度 1200 字节和来自运输层的目的地地址的报文段。网络层则保证将该报文段交付给位于目的主机的运输层。假定在目的主机上能够运行许多网络应用进程。
- 设计可能最简单的运输层协议，该协议将使应用程序数据到达位于目的主机的所希望的进程。假设在目的主机中的操作系统已经为每个运行的应用进程分配了一个 4 字节的端口号。
 - 修改这个协议，使它向目的进程提供一个的“返回地址”。
 - 在你的协议中，该运输层在计算机网络的核心中“必须做任何事”吗？
- R2. 考虑有一个星球，每个人都属于某个六口之家，每个家庭都住在自己的房子里，每个房子都有一个唯一的地址，并且某给定家庭中的每个人有一个独特的名字。假定该星球有一个从源家庭到目的家庭交付信件的邮政服务。该邮件服务要求：①在一个信封中有一封信；②在信封上清楚地写上目的家庭的地址（并且没有别的东西）。假设每个家庭有一名家庭成员代表为家庭中的其他成员收集和分发信件。这些信没有必要提供任何有关信的接收者的指示。
- 使用对上面复习题 R1 的解决方案作为启发，描述家庭成员代表能够使用的协议，以从发送家庭成员向接收家庭成员交付信件。
 - 在你的协议中，该邮政服务必须打开信封并检查信件内容才能提供它的服务吗？
- R3. 考虑在主机 A 和主机 B 之间有一条 TCP 连接。假设从主机 A 传送到主机 B 的 TCP 报文段具有源端口号 x 和目的端口号 y 。对于从主机 B 传送到主机 A 的报文段，源端口号和目的端口号分别是多少？
- R4. 描述应用程序开发者为什么可能选择在 UDP 上运行应用程序而不是在 TCP 上运行的原因。
- R5. 在今天的因特网中，为什么语音和图像流量常常是经过 TCP 而不是经 UDP 发送。（提示：我们寻找的答案与 TCP 的拥塞控制机制没有关系。）
- R6. 当某应用程序运行在 UDP 上时，该应用程序可能得到可靠数据传输吗？如果能，如何实现？
- R7. 假定在主机 C 上的一个进程有一个具有端口号 6789 的 UDP 套接字。假定主机 A 和主机 B 都用目的端口号 6789 向主机 C 发送一个 UDP 报文段。这两台主机的这些报文段在主机 C 都被描述为相同的套接字吗？如果是这样的话，在主机 C 的该进程将怎样知道源于两台不同主机的这两个报文段？
- R8. 假定在主机 C 端口 80 上运行的一个 Web 服务器。假定这个 Web 服务器使用持续连接，并且正在接收来自两台不同主机 A 和 B 的请求。被发送的所有请求都通过位于主机 C 的相同套接字吗？如果它们通过不同的套接字传递，这两个套接字都具有端口 80 吗？讨论和解释之。

3.4 节

- R9. 在我们的 rdt 协议中，为什么需要引入序号？
- R10. 在我们的 rdt 协议中，为什么需要引入定时器？
- R11. 假定发送方和接收方之间的往返时延是固定的并且为发送方所知。假设分组能够丢失的话，在协议 rdt3.0 中，一个定时器仍是必需的吗？试解释之。
- R12. 在配套网站上使用 Go-Back-N（回退 N 步）Java 小程序。
- 让源发送 5 个分组，在这 5 个分组的任何一个到达目的地之前暂停该动画。然后毁掉第一个分组并继续该动画。试描述发生的情况。
 - 重复该实验，只是现在让第一个分组到达目的地并毁掉第一个确认。再次描述发生的情况。
 - 最后，尝试发送 6 个分组。发生了什么情况？

R13. 重复复习题 R12, 但是现在使用 Selective Repeat (选择重传) Java 小程序。选择重传和回退 N 步有什么不同?

3.5 节

R14. 是非判断题:

- 主机 A 经过一条 TCP 连接向主机 B 发送一个大文件。假设主机 B 没有数据发往主机 A。因为主机 B 不能随数据捎带确认, 所以主机 B 将不向主机 A 发送确认。
- 在连接的整个过程中, TCP 的 `rwnd` 的长度决不会变化。
- 假设主机 A 通过一条 TCP 连接向主机 B 发送一个大文件。主机 A 发送但未被确认的字节数不会超过接收缓存的大小。
- 假设主机 A 通过一条 TCP 连接向主机 B 发送一个大文件。如果对于这条连接的一个报文段的序号为 m , 则对于后继报文段的序号将必然是 $m + 1$ 。
- TCP 报文段在它的首部中有一个 `rwnd` 字段。
- 假定在一条 TCP 连接中最后的 `SampleRTT` 等于 1 秒, 那么对于该连接的 `TimeoutInterval` 的当前值必定大于等于 1 秒。
- 假设主机 A 通过一条 TCP 连接向主机 B 发送一个序号为 38 的 4 个字节的报文段。在这个相同的报文段中, 确认号必定是 42。

R15. 假设主机 A 通过一条 TCP 连接向主机 B 发送两个紧接着的 TCP 报文段。第一个报文段的序号为 90, 第二个报文段序号为 110。

- 第一个报文段中有多少数据?
- 假设第一个报文段丢失而第二个报文段到达主机 B。那么在主机 B 发往主机 A 的确认报文中, 确认号应该是多少?

R16. 考虑在 3.5 节中讨论的 Telnet 的例子。在用户键入字符 C 数秒之后, 用户又键入字符 R。那么在用户键入字符 R 之后, 总共发送了多少个报文段, 这些报文段中的序号和确认字段应该填入什么?

3.7 节

R17. 假设两条 TCP 连接存在于一个带宽为 R bps 的瓶颈链路上。它们都要发送一个很大的文件 (以相同方向经过瓶颈链路), 并且两者是同时开始发送文件。那么 TCP 将为每条连接分配什么样的传输速率?

R18. 是非判断题。考虑 TCP 的拥塞控制。当发送方定时器超时, 其 `ssthresh` 的值将被设置为原来值的一半。

R19. 在 3.7 节的“TCP 分岔”讨论中, 对于 TCP 分岔的响应时间, 断言大约是 $4 * RTT_{FE} + RTT_{BE} + \text{处理时间}$ 。评价该断言。

习题

P1. 假设客户 A 向服务器 S 发起一个 Telnet 会话。与此同时, 客户 B 也向服务器 S 发起一个 Telnet 会话。给出下面报文段的源端口号和目的端口号:

- 从 A 向 S 发送的报文段。
- 从 B 向 S 发送的报文段。
- 从 S 向 A 发送的报文段。
- 从 S 向 B 发送的报文段。
- 如果 A 和 B 是不同的主机, 那么从 A 向 S 发送的报文段的源端口号是否可能与从 B 向 S 发送的报文段的源端口号相同?
- 如果它们是同一台主机, 情况会怎么样?

P2. 考虑图 3-5。从服务器返回客户进程的报文流中的源端口号和目的端口号是多少? 在承载运输层报文

段的网络层数据报中，IP 地址是多少？

- P3. UDP 和 TCP 使用反码来计算它们的检验和。假设你有下面 3 个 8 比特字节：01010011、01100110、01110100。这些 8 比特字节和的反码是多少？（注意到尽管 UDP 和 TCP 使用 16 比特的字来计算检验和，但对于这个问题，你应该考虑 8 比特和。）写出所有工作过程。UDP 为什么要用该和的反码，即为什么不直接使用该和呢？使用该反码方案，接收方如何检测出差错？1 比特的差错将可能检测不出来吗？2 比特的差错呢？
- P4. a. 假定你有下列 2 个字节：01011100 和 01100101。这 2 个字节之和的反码是什么？
b. 假定你有下列 2 个字节：11011010 和 01100101。这 2 个字节之和的反码是什么？
c. 对于（a）中的字节，给出一个例子，使得这 2 个字节中的每一个都在一个比特反转时，其反码不会改变。
- P5. 假定某 UDP 接收方对接收到的 UDP 报文段计算因特网检验和，并发现它与承载在检验和字段中的值相匹配。该接收方能够绝对确信没有出现过比特差错吗？试解释之。
- P6. 考虑我们改正协议 rdt2.1 的动机。试说明如图 3-57 所示的接收方与如图 3-11 所示的发送方运行时，接收方可能会引起发送方和接收方进入死锁状态，即双方都在等待不可能发生的事件。



图 3-57 协议 rd2.1 的一个不正确的接收方

- P7. 在 rdt3.0 协议中，从接收方向发送方流动的 ACK 分组没有序号（尽管它们具有 ACK 字段，该字段包括了它们正在确认的分组的序号）。为什么这些 ACK 分组不需要序号呢？
- P8. 画出协议 rdt3.0 中接收方的 FSM。
- P9. 当数据分组和确认分组发生篡改时，给出 rd3.0 协议运行的轨迹。你画的轨迹应当类似于图 3-16 中所用的图。
- P10. 考虑一个能够丢失分组但其最大时延已知的信道。修改协议 rdt2.1，以包括发送方超时和重传机制。非正式地论证：为什么你的协议能够通过该信道正确通信？
- P11. 考虑在图 3-14 中的 rdt2.2 接收方，在状态“等待来自下层的 0”和状态“等待来自下层的 1”中的自转换（即从某状态转换回自身）中生成一个新分组：sndpk = make_pkt(ACK, 1, checksum) 和 sndpk = make_pkt(ACK, 0, checksum)。如果这个动作从状态“等待来自下层的 1”中的自转换中删除，该协议将正确工作吗？评估你的答案。在状态“等待来自下层的 0”中的自转换中删除这个事件将会怎样？[提示：在后一种情况下，考虑如果第一个发送方到接收方的分组损坏的话，将会发生什么情况？]

- P12. rdt3.0 协议的发送方直接忽略（即不采取任何动作）接收到的所有出现差错和确认分组的确认号（acknum）字段中的值有差错的分组。假设在这种情况下，rdt3.0 只是重传当前的数据分组，该协议是否还能正常运行？（提示：考虑在下列情况下会发生什么情况：仅有一个比特差错时；报文没有丢失但能出现定时器过早超时。考虑到当 n 趋于无穷时，第 n 个分组将被发送多少次。）
- P13. 考虑 rdt 3.0 协议。如果发送方和接收方的网络连接能够对报文重排序（即在发送方和接收方之间的媒体上传播的两个报文段能重新排序），那么比特交替协议将不能正确工作（确信你清楚地理解这时它不能正确工作的原因），试画图说明之。画图时把发送方放在左边，接收方放在右边，使时间轴朝下，标出交换的数据报文（D）和确认报文（A）。要标明与任何数据和确认报文段相关的序号。
- P14. 考虑一种仅使用否定确认的可靠数据传输协议。假定发送方只是偶尔发送数据。只用 NAK 的协议是否会比使用 ACK 的协议更好？为什么？现在我们假设发送方要发送大量的数据，并且该端到端连接很少丢包。在第二种情况下，只用 NAK 的协议是否会比使用 ACK 的协议更好？为什么？
- P15. 考虑显示在图 3-17 中的网络跨越国家的例子。窗口长度设置成多少时，才能使该信道的利用率超过 90%？假设分组的长度为 1500 字节（包括首部字段和数据）。
- P16. 假设某应用使用 rdt3.0 作为其运输层协议。因为停等协议具有非常低的信道利用率（显示在网络跨越国家的例子中），该应用程序的设计者让接收方持续回送许多（大于 2）交替的 ACK 0 和 ACK 1，即使对应的数据未到达接收方。这个应用程序设计将能增加信道利用率吗？为什么？该方法存在某种潜在的问题吗？试解释之。
- P17. 考虑两个网络实体 A 和 B，它们由一条完善的双向信道所连接（即任何发送的报文将正确地收到；信道将不会损坏、丢失或重排序分组）。A 和 B 将以交互的方式彼此交付报文：首先，A 必须向 B 交付一个报文，B 然后必须向 A 交付一个报文，接下来 A 必须向 B 交付一个报文，等等。如果一个实体处于它不试图向另一侧交付报文的状态，将存在一个来自上层的类似于 `rdt_send(data)` 调用的事件，它试图向下传送数据以向另一侧传输，来自上层的该调用能够直接忽略对于 `rdt_unable_to_send(data)` 调用，这通知较高层当前不能够发送数据。[注意：做出这种简化的假设，使你不必担心缓存数据。]

对该协议画出 FSM 说明（一个 FSM 用于 A，一个 FSM 用于 B）。注意你不必担心这里的可靠性机制，该问题的要点在于创建反映这两个实体的同步行为的 FSM 说明。应当使用与图 3-9 中协议 rdt1.0 有相同含义的下列事件和动作：`rdt_send(data)`，`packet = make_pkt(data)`，`udt_send(data)`，`rdt_rcv(packet)`，`extract(packet, data)`，`deliver_data(data)`。保证你的协议反映了 A 和 B 之间发送的严格交替。还要保证在你的 FSM 描述中指出 A 和 B 的初始状态。

- P18. 在 3.4.4 节我们学习的一般性 SR 协议中，只要报文可用（如果报文在窗口中），发送方就会不等待确认而传输报文。假设现在我们要求一个 SR 协议，一次发出一对报文，而且只有在知道第一对报文中的一个报文都正确到达后才发送第二对报文。

假设该信道中可能会丢失报文，但报文不会发生损坏和失序。试为报文的单向可靠传输而设计一个差错控制协议。画出发送方和接收方的 FSM 描述。描述在发送方和接收方之间两个方向发送的报文格式。如果你使用了不同于 3.4 节（例如 `udt_send()`、`start_timer()`、`rdt_rcv()` 等）中的任何其他过程调用，详细地阐述这些动作。举例说明（用发送方和接收方的时序踪迹图）你的协议是如何恢复报文丢失的。

- P19. 考虑一种情况，主机 A 想同时向主机 B 和主机 C 发送分组。A 与 B 和 C 是经过广播信道连接的，即由 A 发送的分组通过该信道传送到 B 和 C。假设连接 A、B 和 C 的这个广播信道具有独立的报文丢失和损坏特性（例如，从 A 发出的报文可能被 B 正确接收，但没有被 C 正确接收）。设计一个类似于停等协议的差错控制协议，用于从 A 可靠地传输分组到 B 和 C。该协议使得 A 直到得知 B 和 C 已经正确接收到当前报文，才获取上层交付的新数据。给出 A 和 C 的 FSM 描述。（提示：B 的 FSM 大体上应当与 C 的相同。）同时，给出所使用的报文格式的描述。

- P20. 考虑一种主机 A 和主机 B 要向主机 C 发送报文的情况。主机 A 和 C 通过一条报文能够丢失和损坏（但不重排序）的信道相连接。主机 B 和 C 由另一条（与连接 A 和 C 的信道独立）具有相同性质的信道连接。在主机 C 上的运输层，在向上层交付来自主机 A 和 B 的报文时应当交替进行（即它应当首先交付来自 A 的分组中的数据，然后是来自 B 的分组中的数据，等等）。设计一个类似于停等协议的差错控制协议，以可靠地向 C 传输来自 A 和 B 的分组，同时以前面描述的方式在 C 处交替地交付。给出 A 和 C 的 FSM 描述。（提示：B 的 FSM 大体上应当与 A 的相同。）同时，给出所使用的报文格式的描述。
- P21. 假定我们有两个网络实体 A 和 B。B 有一些数据报文要通过下列规则传给 A。当 A 从其上层得到一个请求，就从 B 获取下一个数据（D）报文。A 必须通过 A—B 信道向 B 发送一个请求（R）报文。仅当 B 收到一个 R 报文后，它才会通过 B—A 信道向 A 发送一个数据（D）报文。A 应当准确地将每份 D 报文的副本交付给上层。R 报文可能会在 A—B 信道中丢失（但不会损坏）；D 报文一旦发出总是能够正确交付。两个信道的时延未知且是变化的。
- 设计一个协议（给出 FSM 描述），它能够综合适当的机制，以补偿会丢包的 A—B 信道，并且实现在 A 实体中向上层传递报文。只采用绝对必要的机制。
- P22. 考虑一个 GBN 协议，其发送方窗口为 4，序号范围为 1024。假设在时刻 t ，接收方期待的下一个有序分组的序号是 k 。假设媒体不会对报文重新排序。回答以下问题：
- 在 t 时刻，发送方窗口内的报文序号可能是多少？论证你的回答。
 - 在 t 时刻，在当前传播回发送方的所有可能报文中，ACK 字段的所有可能值是多少？论证你的回答。
- P23. 考虑 GBN 协议和 SR 协议。假设序号空间的长度为 k ，那么为了避免出现图 3-27 中的问题，对于这两种协议中的每一种，允许的发送方窗口最大为多少？
- P24. 对下面的问题判断是非，并简要地证实你的回答：
- 对于 SR 协议，发送方可能会收到落在其当前窗口之外的分组的 ACK。
 - 对于 GBN 协议，发送方可能会收到落在其当前窗口之外的分组的 ACK。
 - 当发送方和接收方窗口长度都为 1 时，比特交替协议与 SR 协议相同。
 - 当发送方和接收方窗口长度都为 1 时，比特交替协议与 GBN 协议相同。
- P25. 我们曾经说过，应用程序可能选择 UDP 作为运输协议，因为 UDP 提供了（比 TCP）更好的应用层控制，以决定在报文段中发送什么数据和发送时机。
- 应用程序为什么对在报文段中发送什么数据有更多的控制？
 - 应用程序为什么对何时发送报文段有更多的控制？
- P26. 考虑从主机 A 向主机 B 传输 L 字节的大文件，假设 MSS 为 536 字节。
- 为了使得 TCP 序号不至于用完， L 的最大值是多少？前面讲过 TCP 的序号字段为 4 字节。
 - 对于你在（a）中得到的 L ，求出传输此文件要用多长时间？假定运输层、网络层和数据链路层首部总共为 66 字节，并加在每个报文段上，然后经 155Mbps 链路发送得到的分组。忽略流量控制和拥塞控制，使主机 A 能够一个接一个和连续不断地发送这些报文段。
- P27. 主机 A 和 B 经一条 TCP 连接通信，并且主机 B 已经收到了来自 A 的最长为 126 字节的所有字节。假定主机 A 随后向主机 B 发送两个紧接着的报文段。第一个和第二个报文段分别包含了 80 字节和 40 字节的数据。在第一个报文段中，序号是 127，源端口号是 302，目的地端口号是 80。无论何时主机 B 接收到来自主机 A 的报文段，它都会发送确认。
- 在从主机 A 发往 B 的第二个报文段中，序号、源端口号和目的端口号各是什么？
 - 如果第一个报文段在第二个报文段之前到达，在第一个到达报文段的确认中，确认号、源端口号和目的端口号各是什么？
 - 如果第二个报文段在第一个报文段之前到达，在第一个到达报文段的确认中，确认号是什么？
 - 假定由 A 发送的两个报文段按序到达 B。第一个确认丢失了而第二个确认在第一个超时间隔之后到达。画出时序图，显示这些报文段和发送的所有其他报文段和确认。（假设没有其他分组丢失。）对于图上每个报文段，标出序号和数据的字节数量；对于你增加的每个应答，标出确认号。

- P28. 主机 A 和 B 直接经一条 100Mbps 链路连接。在这两台主机之间有一条 TCP 连接。主机 A 经这条连接向主机 B 发送一个大文件。主机 A 能够向它的 TCP 套接字以高达 120Mbps 的速率发送应用数据，而主机 B 能够以最大 50Mbps 的速率从它的 TCP 接收缓存中读出数据。描述 TCP 流量控制的影响。
- P29. 在 3.5.6 节中讨论了 SYN cookie。
- 服务器在 SYNACK 中使用一个特殊的初始序号，这为什么是必要的？
 - 假定某攻击者得知了一台目标主机使用了 SYN cookie。该攻击者能够通过直接向目标发送一个 ACK 分组创建半开或全开连接吗？为什么？
 - 假设某攻击者收集了由服务器发送的大量初始序号。该攻击者通过发送具有初始序号的 ACK，能够引起服务器产生许多全开连接吗？为什么？
- P30. 考虑在 3.6.1 节中显示在第二种情况下的网络。假设发送主机 A 和 B 具有某些固定的超时值。
- 证明增加路由器有限缓存的长度可能减小吞吐量 (λ_{out})。
 - 现在假设两台主机基于路由器的缓存时延，动态地调整它们的超时值（像 TCP 所做的那样）。增加缓存长度将有助于增加吞吐量吗？为什么？
- P31. 假设测量的 5 个 SampleRTT 值（参见 3.5.3 节）是 106ms、120ms、140ms、90ms 和 115ms。在获得了每个 SampleRTT 值后计算 EstimatedRTT，使用 $\alpha = 0.125$ 并且假设在刚获得前 5 个样本之后 EstimatedRTT 的值为 100ms。在获得每个样本之后，也计算 DevRTT，假设 $\beta = 0.25$ ，并且假设在刚获得前 5 个样本之后 DevRTT 的值为 5ms。最后，在获得这些样本之后计算 TCP TimeoutInterval。
- P32. 考虑 TCP 估计 RTT 的过程。假设 $\alpha = 0.1$ ，令 SampleRTT_{t1} 设置为最新样本 RTT，令 SampleRTT_{t2} 设置为下一个最新样本 RTT，等等。
- 对于一个给定的 TCP 连接，假定 4 个确认报文相继到达，带有 4 个对应的 RTT 值： SampleRTT_{t4} 、 SampleRTT_{t3} 、 SampleRTT_{t2} 和 SampleRTT_{t1} 。根据这 4 个样本 RTT 表示 EstimatedRTT。
 - 将你得到的公式一般化到 n 个 RTT 样本的情况。
 - 对于在 (b) 中得到的公式，令 n 趋于无穷。试说明为什么这个平均过程被称为指数移动平均。
- P33. 在 3.5.3 节中，我们讨论了 TCP 的往返时间的估计。TCP 避免测量重传报文段的 SampleRTT，对此你有何看法？
- P34. 3.5.4 节中的变量 SendBase 和 3.5.5 节中的变量 LastByteRcvd 之间有什么关系？
- P35. 3.5.5 节中的变量 LastByteRcvd 和 3.5.4 节中的变量 y 之间有什么关系？
- P36. 在 3.5.4 节中，我们看到 TCP 直到收到 3 个冗余 ACK 才执行快速重传。你对 TCP 设计者没有选择在收到对报文段的第一个冗余 ACK 后就快速重传有何看法？
- P37. 比较 GBN、SR 和 TCP（无延时的 ACK）。假设对所有 3 个协议的超时值足够长，使得 5 个连续的数据报文段及其对应的 ACK 能够分别由接收主机（主机 B）和发送主机（主机 A）收到（如果在信道中无丢失）。假设主机 A 向主机 B 发送 5 个数据报文段，并且第二个报文段（从 A 发送）丢失。最后，所有 5 个数据报文段已经被主机 B 正确接收。
- 主机 A 总共发送了多少报文段和主机 B 总共发送了多少 ACK？它们的序号是什么？对所有 3 个协议回答这个问题。
 - 如果对所有 3 个协议超时值比 5RTT 长得多，则哪个协议在最短的时间间隔中成功地交付所有 5 个数据报文段？
- P38. 在图 3-53 中的 TCP 描述中，阈值 ssthresh 的值在几个地方被设置为 $\text{ssthresh} = \text{cwnd}/2$ ，并且当出现一个丢包事件时，ssthresh 的值被设置为窗口长度的一半。当出现丢包事件时，发送方发送的速率，每个 RTT 必须大约等于 cwnd 报文段吗？解释你的答案。如果你的回答是没有，你能建议一种不同的方式，进行 ssthresh 设置吗？
- P39. 考虑图 3-46b。如果 λ'_{in} 增加超过了 $R/2$ ， λ_{out} 能够增加超过 $R/3$ 吗？试解释之。现在考虑图 3-46c。假定一个分组从路由器到接收方平均转发两次的话，如果 λ'_{in} 增加超过 $R/2$ ， λ_{out} 能够增加超过 $R/4$ 吗？试解释之。

P40. 考虑图 3-58。假设 TCP Reno 是一个经历如上所示行为的协议，回答下列问题。在各种情况中，简要地论证你的回答。

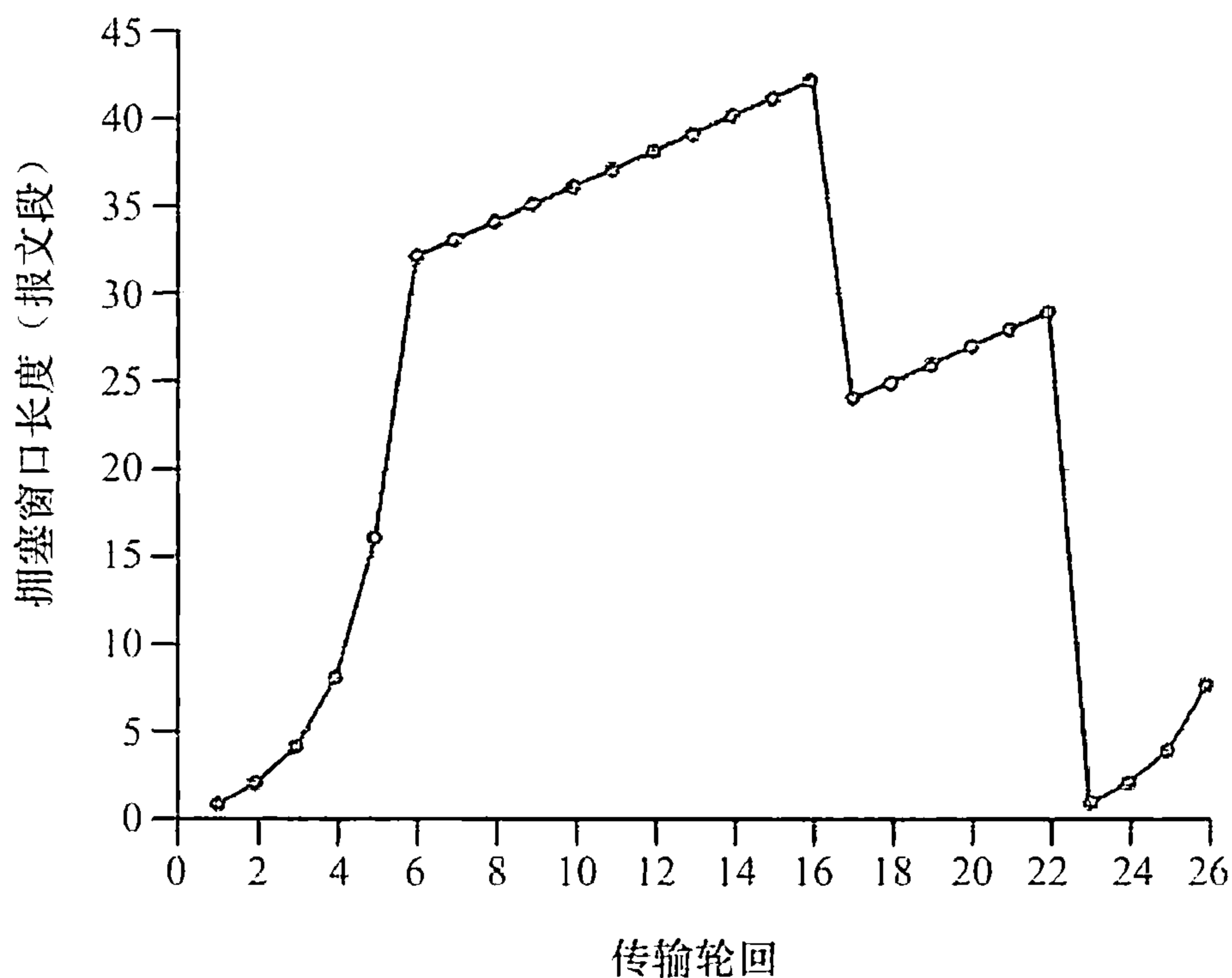


图 3-58 TCP 窗口长度作为时间的函数

- 指出 TCP 慢启动运行时的时间间隔。
 - 指出 TCP 拥塞避免运行时的时间间隔。
 - 在第 16 个传输轮回之后，报文段的丢失是根据 3 个冗余 ACK 还是根据超时检测出来的？
 - 在第 22 个传输轮回之后，报文段的丢失是根据 3 个冗余 ACK 还是根据超时检测出来的？
 - 在第 1 个传输轮回里，sssthresh 的初始值设置为多少？
 - 在第 18 个传输轮回里，sssthresh 的值设置为多少？
 - 在第 24 个传输轮回里，sssthresh 的值设置为多少？
 - 在哪个传输轮回内发送第 70 个报文段？
 - 假定在第 26 个传输轮回后，通过收到 3 个冗余 ACK 检测出有分组丢失，拥塞的窗口长度和 sssthresh 的值应当是多少？
 - 假定使用 TCP Tahoe（而不是 TCP Reno），并假定在第 16 个传输轮回收到 3 个冗余 ACK。在第 19 个传输轮回，sssthresh 和拥塞窗口长度是什么？
 - 再次假设使用 TCP Tahoe，在第 22 个传输轮回有一个超时事件。从第 17 个传输轮回回到第 22 个传输轮回（包括这两个传输轮回），一共发送了多少分组？
- P41. 参考图 3-56，该图描述了 TCP 的 AIMD 算法的收敛特性。假设 TCP 不采用乘性减，而是采用按某一常量减小窗口。所得的 AIAD 算法将收敛于一种平等共享算法吗？使用类似于图 3-56 中的图来证实你的结论。
- P42. 在 3.5.4 节中，我们讨论了在发生超时事件后将超时间隔加倍。为什么除了这种加倍超时间隔机制外，TCP 还需要基于窗口的拥塞控制机制（如在 3.7 节中学习的那种机制）呢？
- P43. 主机 A 通过一条 TCP 连接向主机 B 发送一个很大的文件。在这条连接上，不会出现任何分组丢失和定时器超时。主机 A 与因特网连接链路的传输速率表示为 R bps。假设主机 A 上的进程能够以 S bps 的速率向 TCP 套接字发送数据，其中 $S = 10 \times R$ 。进一步假设 TCP 的接收缓存足够大，能够容纳整个文件，并且发送缓存只能容纳这个文件的百分之一。如何防止主机 A 上的进程连续地向 TCP 套接字以速率 S bps 传送数据呢？还是用 TCP 流量控制呢？还是用 TCP 拥塞控制？或者用其他措施？阐述其理由。
- P44. 考虑从一台主机经一条没有丢包的 TCP 连接向另一台主机发送一个大文件。
- 假定 TCP 使用不具有慢启动的 AIMD 进行拥塞控制。假设每当收到一批 ACK 时，cwnd 增加 1 个 MSS，并且假设往返时间大约恒定，cwnd 从 6MSS 增加到 12MSS 要花费多长时间（假设没有丢包事件）？

b. 对于该连接，到时间 $= 6RTT$ ，其平均吞吐量是多少（根据 MSS 和 RTT）？

P45. 回想 TCP 吞吐量的宏观描述。在连接速率从 $W/(2 \times RTT)$ 变化到 W/RTT 的周期内，只丢失了一个分组（在该周期的结束）。

a. 证明其丢包率（分组丢失的比率）等于：

$$L = \text{丢包率} = \frac{1}{\frac{3}{8}W^2 + \frac{3}{4}W}$$

b. 如果一条连接的丢包率为 L ，使用上面的结果，则它的平均速率近似由下式给出：

$$\text{平均速率} \approx \frac{1.22 * \text{MSS}}{RTT \sqrt{L}}$$

P46. 考虑仅有一条单一的 TCP（Reno）连接使用一条 10Mbps 链路，且该链路没有缓存任何数据。假设这条链路是发送主机和接收主机之间的唯一拥塞链路。假定某 TCP 发送方向接收方有一个大文件要发送，而接收方的接收缓存比拥塞窗口要大得多。我们也做下列假设：每个 TCP 报文段长度为 1500 字节；该连接的双向传播时延是 150ms；并且该 TCP 连接总是处于拥塞避免阶段，即忽略了慢启动。

a. 这条 TCP 连接能够取得的最大窗口长度（以报文段计）是多少？

b. 这条 TCP 连接的平均窗口长度（以报文段计）和平均吞吐量（以 bps 计）是多少？

c. 这条 TCP 连接在从丢包恢复后，再次到达其最大窗口要经历多长时间？

P47. 考虑在前面习题中所描述的场景。假设 10Mbps 链路能够缓存有限个报文段。试论证为了使该链路总是忙于发送数据，我们将要选择缓存长度，使得其至少为发送方和接收方之间链路速率 C 与双向传播时延之积。

P48. 重复习题 46，但用一条 10Gbps 链路代替 10Mbps 链路。注意到在对 c 部分的答案中，应当认识到在从丢包恢复后，拥塞窗口长度到达最大窗口长度将需要很长时间。给出解决该问题的基本思路。

P49. 令 T （用 RTT 度量）表示一条 TCP 连接将拥塞窗口从 $W/2$ 增加到 W 所需的时间间隔，其中 W 是最大的拥塞窗口长度。论证 T 是 TCP 平均吞吐量的函数。

P50. 考虑一种简化的 TCP 的 AIMD 算法，其中拥塞窗口长度用报文段的数量来度量，而不是用字节度量。在加性增中，每个 RTT 拥塞窗口长度增加一个报文段。在乘性减中，拥塞窗口长度减小一半（如果结果不是一个整数，向下取整到最近的整数）。假设两条 TCP 连接 C1 和 C2，它们共享一条速率为每秒 30 个报文段的单一拥塞链路。假设 C1 和 C2 均处于拥塞避免阶段。连接 C1 的 RTT 是 50ms，连接 C2 的 RTT 是 100ms。假设当链路中的数据速率超过了链路的速率时，所有 TCP 连接经受数据报文段丢失。

a. 如果在时刻 t_0 ，C1 和 C2 具有 10 个报文段的拥塞窗口，在 1000ms 后它们的拥塞窗口为多长？

b. 经长时间运行，这两条连接将取得共享该拥塞链路的相同的带宽吗？

P51. 考虑在前面习题中描述的网络。现在假设两条 TCP 连接 C1 和 C2，它们具有相同的 100ms RTT。假设在时刻 t_0 ，C1 的拥塞窗口长度为 15 个报文段，而 C2 的拥塞窗口长度是 10 个报文段。

a. 在 2200ms 后，它们的拥塞窗口长度为多长？

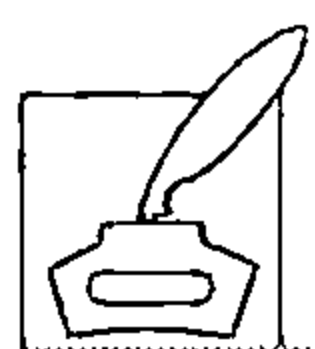
b. 经长时间运行，这两条连接将取得共享该拥塞链路的相同的带宽吗？

c. 如果这两条连接在相同时间达到它们的最大窗口长度，并在相同时间达到它们的最小窗口长度，我们说这两条连接是同步的。经长时间运行，这两条连接将最终变得同步吗？如果是，它们的最大窗口长度是多少？

d. 这种同步将有助于改善共享链路的利用率吗？为什么？给出打破这种同步的某种思路。

P52. 考虑修改 TCP 的拥塞控制算法。不使用加性增，使用乘性增。无论何时某 TCP 收到一个合法的 ACK，就将其窗口长度增加一个小正数 a ($0 < a < 1$)。求出丢包率 L 和最大拥塞窗口 W 之间的函数关系。论证：对于这种修正的 TCP，无论 TCP 的平均吞吐量如何，一条 TCP 连接将其拥塞窗口长度从 $W/2$ 增加到 W ，总是需要相同的时间。

- P53. 在 3.7 节对 TCP 未来的讨论中，我们注意到为了取得 10Gbps 的吞吐量，TCP 仅能容忍 2×10^{-10} 的报文段丢失率（或等价于每 5 000 000 000 个报文段有一个丢包事件）。给出针对 3.7 节中给定的 RTT 和 MSS 值的对 2×10^{-10} 值的推导。如果 TCP 需要支持一条 100Gbps 的连接，所能容忍的丢包率是多少？
- P54. 在 3.7 节中对 TCP 拥塞控制的讨论中，我们隐含地假定 TCP 发送方总是有数据要发送。现在考虑下列情况，某 TCP 发送方发送大量数据，然后在 t_1 时刻变得空闲（因为它没有更多的数据要发送）。TCP 在相对长的时间内保持空闲，然后在 t_2 时刻要发送更多的数据。当 TCP 在 t_2 开始发送数据时，让它使用在 t_1 时刻的 cwnd 和 ssthresh 值，将有什么样的优点和缺点？你建议使用什么样的方法？为什么？
- P55. 在这个习题中我们研究是否 UDP 或 TCP 提供了某种程度的端点鉴别。
- 考虑一台服务器接收到在一个 UDP 分组中的请求并对该请求进行响应（例如，如由 DNS 服务器所做的那样）。如果一个具有 IP 地址 X 的客户用地址 Y 进行哄骗的话，服务器将向何处发送它的响应？
 - 假定一台服务器接收到具有 IP 源地址 Y 的一个 SYN，在用 SYNACK 响应之后，接收一个具有 IP 源地址 Y 和正确确认号的 ACK。假设该服务器选择了一个随机初始序号并且没有“中间人”，该服务器能够确定该客户的确位于 Y 吗？（并且不在某个其他哄骗为 Y 的地址 X。）
- P56. 在这个习题中，我们考虑由 TCP 慢启动阶段引入的时延。考虑一个客户和一个 Web 服务器直接连接到速率 R 的一条链路。假定该客户要取回一个对象，其长度正好等于 $15S$ ，其中 S 是最大段长度（MSS）。客户和服务器的往返时间表示为 RTT（假设为常数）。忽略协议首部，确定在下列情况下取回该对象的时间（包括 TCP 连接创建）：
- $4S/R > S/R + RTT > 2S/R$
 - $S/R + RTT > 4S/R$
 - $S/R > RTT$

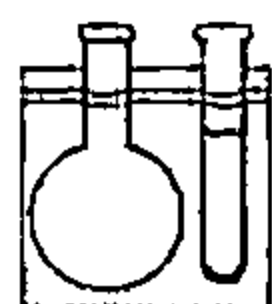


编程作业

实现一个可靠运输协议

在这个编程作业实验中，你将要编写发送和接收运输层的代码，以实现一个简单的可靠数据运输协议。这个实验有两个版本，即比特交替协议版本和 GBN 版本。这个实验应当是有趣的，因为你的实现将与实际情况下所要求的差异很小。

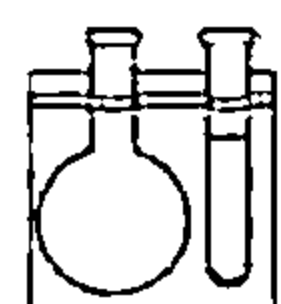
因为可能没有你能够修改其操作系统的独立机器，你的代码将不得不在模拟的硬件/软件环境中执行。然而，为你提供例程的编程接口（即从上层和下层调用你的实体的代码），非常类似于在实际 UNIX 环境中做那些事情的接口。（实际上，在本编程作业中描述的软件接口比起许多教科书中描述的无限循环的发送方和接收方要真实得多。）停止和启动定时器也是模拟的，定时器中断将激活你的定时器处理例程。



Wireshark 实验：探究 TCP

在这个实验中，你将使用 Web 浏览器访问来自某 Web 服务器的一个文件。如同在前面的 Wireshark 实验中一样，你将使用 Wireshark 来俘获到达你计算机的分组。与前面实验不同的是，你也能够从该 Web 服务器下载一个 Wireshark 可读的分组踪迹，记载你从服务器下载文件的过程。在这个服务器踪迹文件里，你将发现自己访问该 Web 服务器所产生的分组。你将分析客户端和服务端踪迹文件，以探究 TCP 的方方面面。特别是你将评估在你的计算机与该 Web 服务器之间 TCP 连接的性能。你将跟踪 TCP 窗口行为、推断分组丢失、重传、流控和拥塞控制行为并估计往返时间。

与所有的 Wireshark 实验一样，该实验的全面描述能够在本书 Web 站点 <http://www.awl.com/kurose-ross> 上找到。



Wireshark 实验：探究 UDP

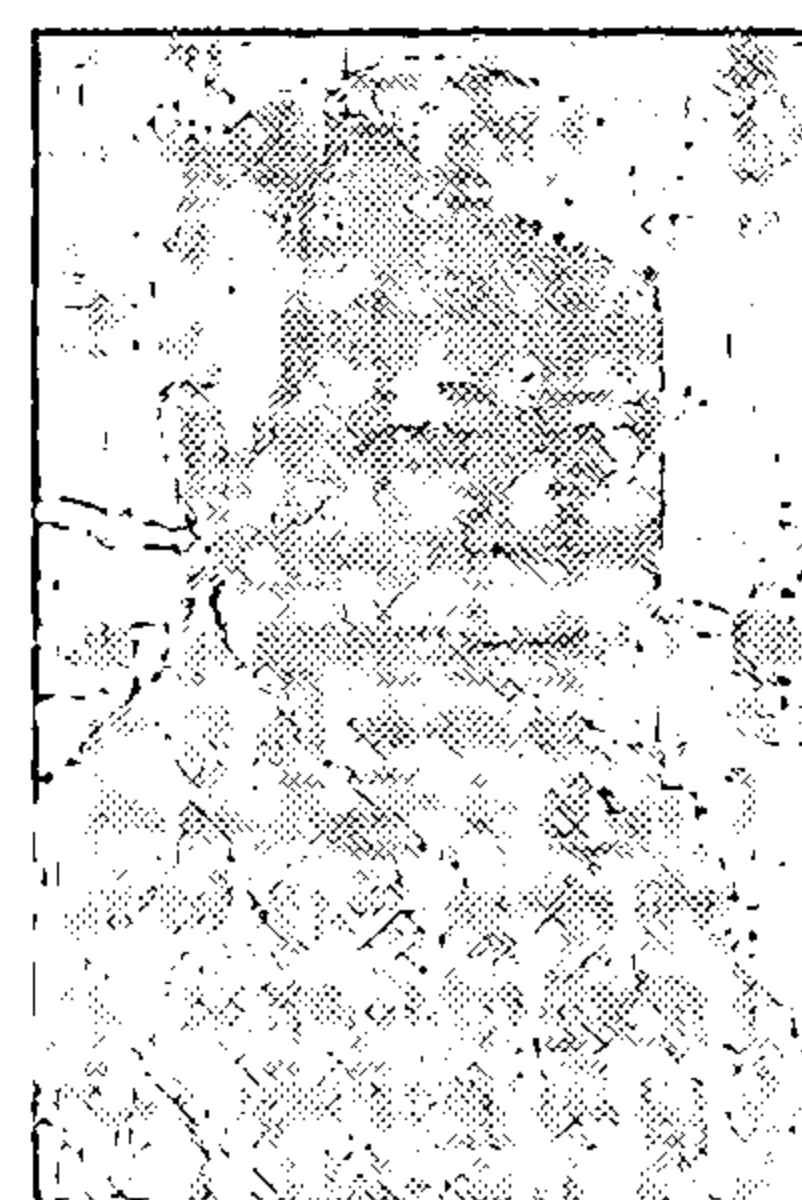
在这个简短实验中，你将进行分组俘获并分析那些使用 UDP 的你喜爱的应用程序（例如，DNS 或如 Skype 这样的多媒体应用）。如我们在 3.3 节中所学的那样，UDP 是一种简单的、不提供不必要服务的运输协议。在这个实验中，你将研究在 UDP 报文段中的各首部字段以及检验和计算。

与所有的 Wireshark 实验一样，该实验的全面描述能够在本书 Web 站点 <http://www.awl.com/kurose-ross> 上找到。



人物专访

Van Jacobson 现在是 PARC 的高级研究员。在此之前，他是分组设计（Packet Design）组织的首席科学家和共同创建者。再往前，他是思科公司的首席科学家。在加入思科之前，他是劳伦兹伯克利国家实验室的网络研究组的负责人，并在加州大学伯克利分校和斯坦福大学任教。Van 于 2001 年因其在通信网络领域的贡献而获得 ACM SIGCOMM 终身成就奖，于 2002 年因其“对网络拥塞的理解和成功研制用于因特网的拥塞控制机制”而获得 IEEE Kobayashi 奖。他于 2004 年当选为美国国家工程院院士。



Van Jacobson

- 请描述在你的职业生涯中从事的一个或两个最为令人兴奋的项目。工作中最大的挑战是什么？

学校教会我们许多寻找答案的方式。在每个我致力于的感兴趣的问题中，艰巨的任务是找到正确的问题。当 Mike Karels 和我开始关注 TCP 拥塞时，我们花费数月凝视协议和分组踪迹，询问“为什么它会失效？”。有一天在 Mike 的办公室，我们中的一个说：“我无法弄明白它失效的原因是因为我不理解它究竟如何开始运转的。”这导致提出了一个正确问题，它迫使我们弄明白使 TCP 运转的“ack 计时”。从那以后，其他东西就容易了。

- 从更为一般意义讲，您认为网络和因特网未来将向何处去？

对于大多数人来说，Web 是因特网。对此，网络奇才们将会善意地窃笑，因为我们知道 Web 是一个运行在因特网上的应用程序，但要是以上说法正确又该如何呢？因特网使得主机对之间能够进行交谈。Web 用于分布信息的生产和消耗。“信息传播”是一种非常一般意义上的通信，而“成对交谈”只是其中一个极小的子集。我们需要向更大的范围进发。今天的网络以点到点连线的方式处理广播媒体（无线电、PON 等）。那是极为低效的。经过指头敲击或智能手机，遍及全世界的每秒兆兆（ 10^{12} ）比特的数据正在交换，但我们不知道如何将其作为“网络”处理。ISP 正在忙于建立缓存和 CDN，以可扩展地分发视频和音频。缓存是该解决方案的必要部分，但今天的网络缺乏这个部分。从信息论、排队论或流量理论直到因特网协议规范，都告诉我们如何建造和部署它。我认为并希望在未来几年中，网络将演化为包含多得多的通信愿景，支撑 Web 的运行。

- 什么人给你的职业带来灵感？

当我还在研究生院时，Richard Feynman 访问了学校并做了学术报告。他讲到了一些量子理论知识，使我整学期都在努力理解该理论，他的解释非常简单和明白易懂，使得那些对我而言难以理解的东西变得显而易见和不可避免。领会和表达复杂世界背后的简单性的能力是给我的罕见和绝妙的礼物。

- 对于希望从事计算机科学和网络的学生，您的建议是什么？

网络是奇妙的领域，计算机和网络对社会的影响，也许比自有文字记载以来的任何发明都大。网络本质上是有关连接的东西，研究它有助于你进行智能连接：蚁群搜索和蜜蜂舞蹈显示了协议设计好于 RFC，流量拥挤或人们离开挤满人的体育馆是拥塞的要素，在感恩节暴风雪后寻找航班返回学校的学生们是动态路由选择的核心。如果你对许多东西感兴趣，并且要对此干点事，很难想象还有什么比网络更好的领域了。

网 络 层

在前一章中我们学习了运输层依赖于网络层的主机到主机的通信服务，提供各种形式的进程到进程的通信。我们也学习了运输层工作时不具备任何有关网络层实际实现这种服务的知识。因此也许你现在想知道，这种主机到主机通信服务的真实情况是什么？是什么使得它工作起来的呢？

在本章中我们将学习网络层实际是怎样实现主机到主机的通信服务的。我们将看到，与运输层和应用层不同的是，在网络中的每一台主机和路由器中都有一个网络层部分。正因如此，网络层协议是协议栈中最具挑战性（因而也是最有趣）的部分。

网络层也是协议栈中最复杂的层次之一，因此我们将在这里涉及大量的知识。我们的学习从网络层的概述和它能够提供的服务开始。我们将仔细考察两种用于构造网络层分组交付的方法，即数据报模式和虚电路模式，并且理解编址在传递分组到目的主机所起的重要作用。

在本章中，我们将对网络层的**转发**（forwarding）功能和**路由选择**（routing）功能做重要区分。转发涉及分组在单一的路由器中从一条入链路到一条出链路的传送。路由选择涉及一个网络的所有路由器，它们经路由选择协议共同交互，以决定分组从源到目的地结点所采用的路径。当你继续本章学习时，心中要记住这个区别。

为了加深对分组转发的理解，我们将“进入”路由器内部来观察它的硬件体系结构和组织。接下来我们将观察在因特网中的分组转发，以及令人称颂的网际协议（IP）。我们将研究网络层编址和 IPv4 的数据报格式。然后我们将探讨网络地址转换（NAT）、数据报分段、因特网控制报文协议（ICMP）和 IPv6。

然后我们将注意力转向网络层的路由选择功能。我们将看到路由选择算法的任务是决定从发送方到接收方的好的路径（等价地为路由）。我们将首先学习路由选择算法的理论，关注两种最为流行的算法类型：链路状态和距离矢量算法。因为路由选择算法的复杂性随着网络路由器数量的增加会有相当大的增长，因此我们也会关注等级制路由选择方法。当我们涉及因特网的自治系统内部的路由选择协议（RIP、OSPF 和 IS-IS）和因特网的自治系统之间的路由选择协议（BGP）时，我们将看到理论是如何付诸实践的。最后我们讨论广播和多播路由选择。

总的来说，本章有 3 个主要部分：第一部分（4.1 节和 4.2 节）涉及了网络层功能和服务；第二部分（4.3 节和 4.4 节）涉及了转发；最后第三部分（4.5 节到 4.7 节）涉及了路由选择。

4.1 概述

图 4-1 显示了一个具有 H1 和 H2 两台主机和在 H1 与 H2 之间的路径上有几台路由器的简单网络。假设 H1 正在向 H2 发送信息，考虑这些主机与中间路由器的网络层所起的作用。H1 中的网络层取得来自于 H1 运输层的报文段，将每个报文段封装成一个数据报（即一个网络层的分组），然后将该数据报向相邻路由器 R1 发送。在接收方主机 H2，网络

层接收来自相邻路由器 R2 的数据报，提取出运输层报文段，并将其向上交付给 H2 的运输层。路由器的主要作用便是将数据报从入链路转发到出链路。注意到图 4-1 中所示路由器具有截断的协议栈，即没有网络层以上的部分，因为（除了控制目的外）路由器不运行我们在第 2、3 章学习过的应用层和运输层协议。

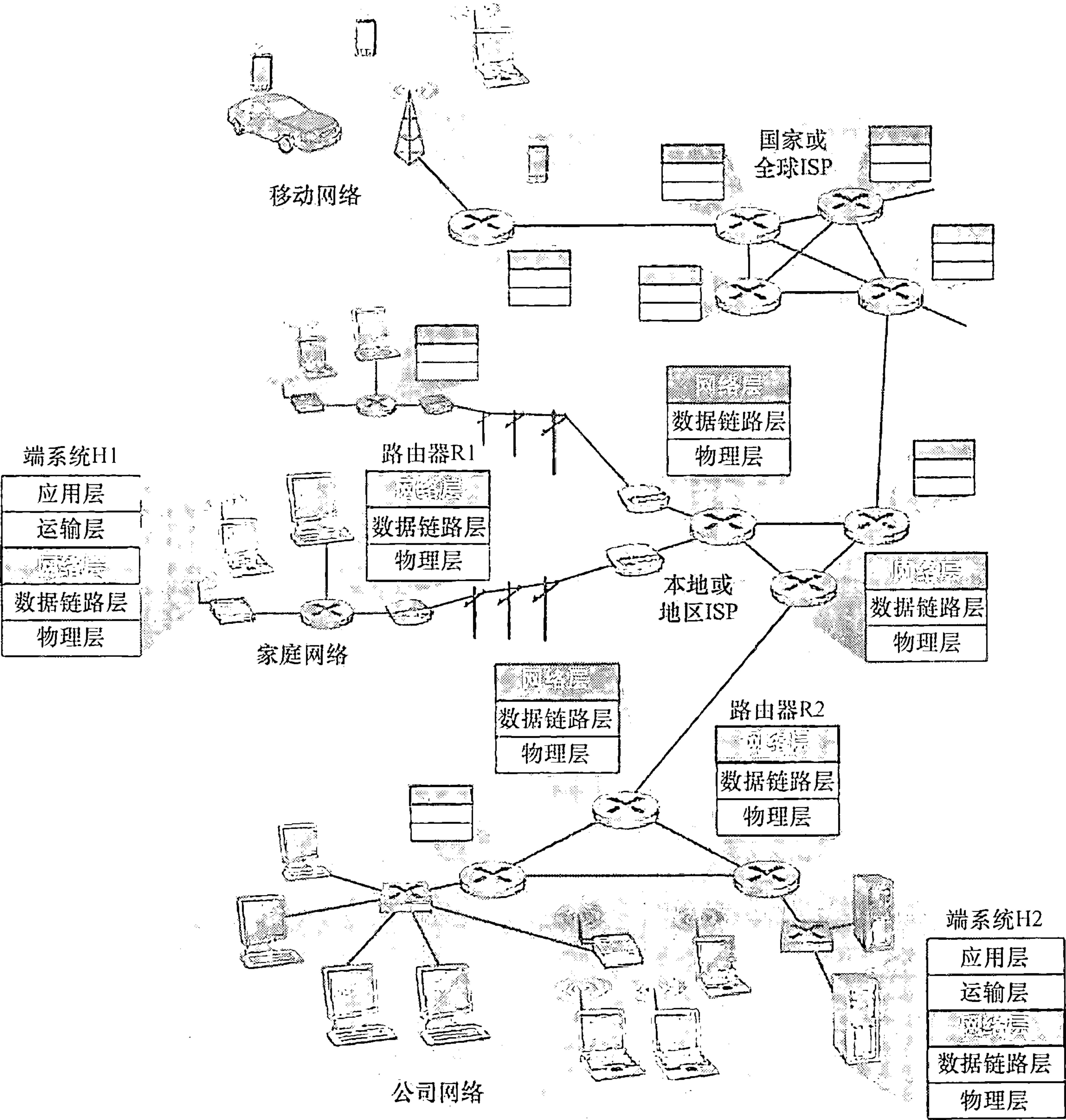


图 4-1 网络层

4.1.1 转发和路由选择

网络层的作用从表面上看极为简单，即将分组从一台发送主机移动到一台接收主机。为此，需要两种重要的网络层功能：

- 转发。当一个分组到达路由器的一条输入链路时，路由器必须将该分组移动到适当的输出链路。例如，来自主机 H1 到路由器 R1 的一个分组，必须向在 H2 路径上的下一台路由器转发。在 4.3 节中，我们将深入路由器内部观察，考察分组在路由器中是如何实际从一条输入链路转发到一条输出链路的。

- 路由选择。当分组从发送方流向接收方时，网络层必须决定这些分组所采用的路由或路径。计算这些路径的算法被称为路由选择算法（routing algorithm）。例如，一个路由选择算法将决定分组从 H1 到 H2 流动所遵循的路径。

在讨论网络层时，许多作者经常互换地使用转发和路由选择这两个术语。我们在本书中将更为精确地使用这些术语。转发是指将分组从一个输入链路接口转移到适当的输出链路接口的路由器本地动作。路由选择是指网络范围的过程，以决定分组从源到目的地所采取的端到端路径。用驾驶的例子进行类比，考虑在 1.3.1 节中旅行者所历经的从宾夕法尼亚州到佛罗里达州的行程。在这个行程中，该驾驶员到佛罗里达州经过了途中的许多立交桥。我们能够认为转发就像通过单个立交桥的过程：一辆汽车沿道路上进入立交桥的一个入口，并且决定应当走哪条路来离开该立交桥。我们能够认为路由选择是规划从费城到佛罗里达州行程的过程：在着手行程之前，驾驶员查阅地图并在许多可能的路径中选择一条，其中每条路径都由一系列经立交桥连接的路段组成。

每台路由器具有一张转发表（forwarding table）。路由器通过检查到达分组首部字段的值来转发分组，然后使用该值在该路由器的转发表中索引查询。存储在转发表项中的该首部的值指出了该分组将被转发的路由器的输出链路接口。分组首部中的该值可能是该分组的目的地地址或该分组所属连接的指示，这取决于网络层协议。图 4-2 提供了一个例子。在图 4-2 中，一个首部字段值为 0111 的分组到达路由器。路由器在它的转发表中索引，决定该分组的输出链路接口是接口 2。然后路由器在内部将该分组转发到接口 2。在 4.3 节中我们深入路由器内部，更为详细地研究这种转发功能。

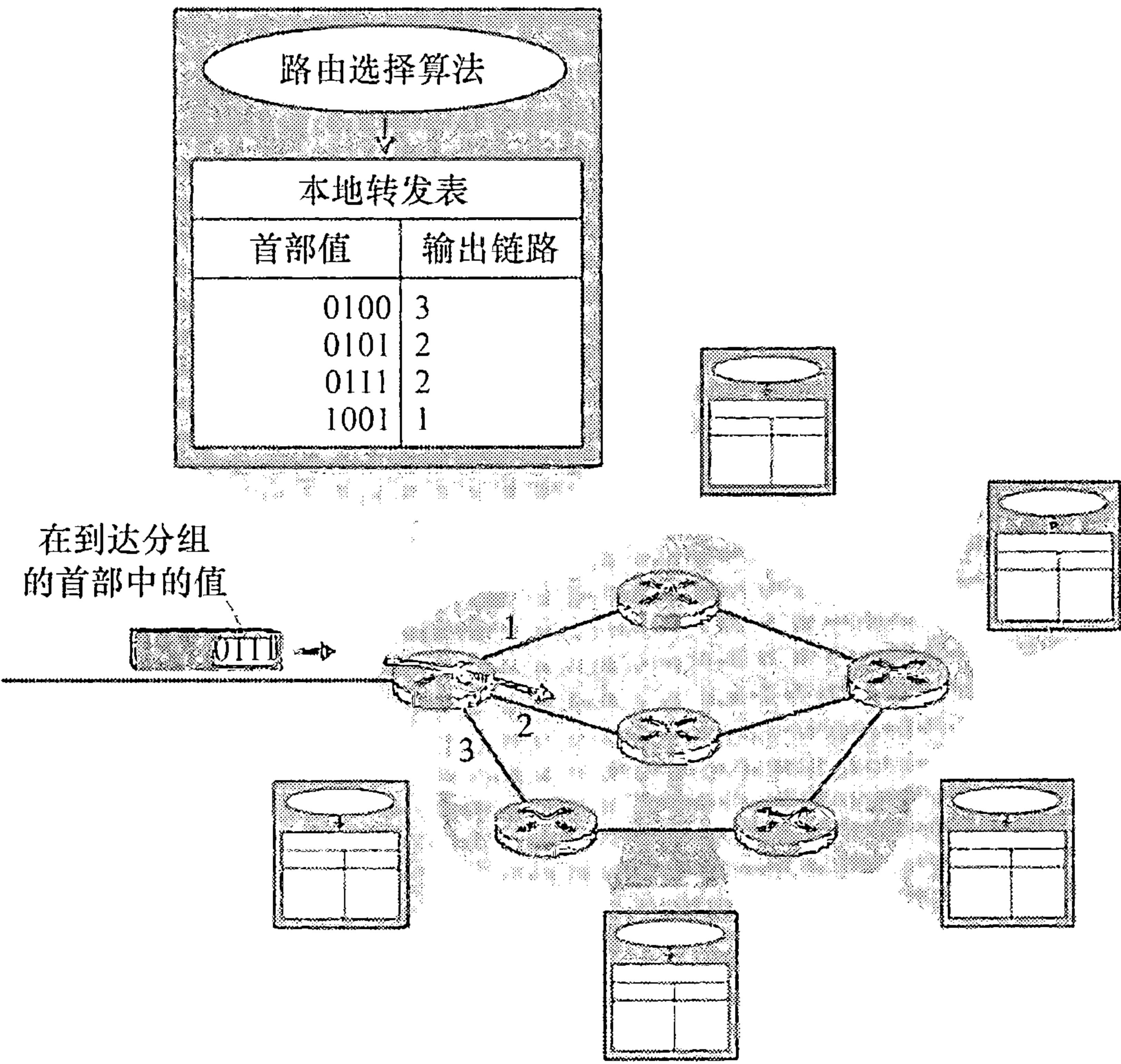


图 4-2 路由选择算法决定转发表中的值

你也许现在想知道路由器中的转发表是如何配置的。这是一个关键问题，它揭示了路由选择和转发间的重要相互作用关系。如图 4-2 中所示，路由选择算法决定了插入路由器的转发表中的值。路由选择算法可能是集中式的（例如，算法在某个中心场点执行，并向

每台路由器下载路由选择信息)，或是分布式的（即，使用运行在每台路由器上的分布式路由选择算法的一部分）。在任何一种情况下，都是路由器接收路由选择协议报文，该信息被用于配置其转发表。通过考虑网络中的一种假想情况（不真实的，但技术上是可行的），其中所有的转发表是由人类网络操作员直接配置而在路由器中物理地存在，转发和路由选择功能的区别和不同能被进一步说明。在这种情况下，不需要任何路由选择协议！当然，该人类操作员将需要彼此交互，以确保该转发表配置得能使分组到达它们想要到达的目的地。也很可能出现下列现象：人工配置更容易出错，并且对于网络拓扑的变化，响应起来比路由选择协议慢。因此，我们庆幸所有网络具有转发和路由选择功能。

当我们讨论术语时，需要指出经常交互使用的两个其他术语，而我们将要更为小心地使用它们。我们将约定术语分组交换机是指一台通用分组交换设备，它根据分组首部字段中的值，从输入链路接口到输出链路接口转移分组。某些分组交换机称为链路层交换机（link-layer switches）（在第5章仔细学习），基于链路层字段中的值做转发决定。其他分组交换机称为路由器（router），基于网络层字段中的值做转发决定。（为了全面理解这种重要区别，你可能要回顾1.5.2节，在那里我们讨论了网络层数据报和链路层帧及其关系。）市场销售资料经常将具有以太网接口的路由器称为“三层交换机”，但它们实际上是三层设备，混淆了相关概念。因为在本章中我们关注的是网络层，所以我们使用术语路由器代替分组交换机。当我们谈论虚电路网络中的分组交换机时，我们甚至将使用词汇路由器（很快将讨论）。

连接建立

我们刚才说过网络层有两个重要的功能，转发和路由选择。但我们很快将看到在某些计算机网络中，实际上有第三种重要的网络功能，即**连接建立**（connection setup）。回想我们学习TCP时，当数据能从发送方流向接收方之前，需要三次握手。这允许发送方和接收方建立所需的状态信息（例如，序号和初始流控制窗口长度）。以类似的方式，某些网络层体系结构如ATM、帧中继、MPLS（我们将在5.8节学习），要求从源到目的地沿着所选择的路径彼此握手，以便在给定源到目的地连接中的网络层数据分组能够开始流动之前建立起状态。在网络层，该过程被称为连接建立。我们将在4.2节中仔细学习连接建立。

4.1.2 网络服务模型

在钻研网络层之前，我们将以开阔的视野来考虑网络层可能提供的不同类型的服务。当位于发送主机的运输层向网络传输分组时（即在发送主机中将分组向下交给网络层），运输层能够指望网络层将该分组交付给目的地吗？当发送多个分组时，它们会按发送顺序按序交付给接收主机的运输层吗？发送两个连续分组的时间间隔与接收到这两个分组的时间间隔相同吗？网络层会提供关于网络中拥塞的反馈信息吗？在发送主机与接收主机中连接运输层的通道的抽象视图（特性）是什么？对这些问题和其他问题的答案由网络层提供的服务模型所确定。**网络服务模型**（network service model）定义了分组在发送与接收端系统之间的端到端运输特性。

我们现在考虑网络层能够提供的某些可能的服务。在发送主机中，当运输层向网络层传递一个分组时，能由网络层提供的特定服务包括：

- 确保交付。该服务确保分组将最终到达其目的地。

- 具有时延上界的确保交付。该服务不仅确保分组的交付，而且在特定的主机到主机时延上界内（例如在 100ms 内）交付。

此外，能够为给定的源和目的地之间的分组流提供下列服务：

- 有序分组交付。该服务确保分组以它们发送的顺序到达目的地。
- 确保最小带宽。这种网络层服务模仿在发送和接收主机之间的一条特定比特率（例如 1Mbps）的传输链路的行为。只要发送主机以低于特定比特率的速率传输比特（作为分组的组成部分），则分组不会丢失，且每个分组会在预定的主机到主机时延内到达（例如在 40ms 内）。
- 确保最大时延抖动。该服务确保位于发送方的两个相继分组之间的时间量等于在目的地接收到它们之间的时间量（或这种间隔的变化不超过某些特定的值）。
- 安全性服务。使用仅由源和目的主机所知晓的一个秘密会话密钥，在源主机中的网络层能够加密向目的主机发送的所有数据报负载。在目的主机中的网络层则能够负责解密该负载。使用这种服务，能够向源和目的主机之间的所有运输层报文段（TCP 和 UDP）提供机密性。除了机密性以外，网络层能够提供数据完整性和源鉴别服务。

这只是网络层能够提供的部分服务的列表，有无数种可能的服务变种。

因特网的网络层提供了单一的服务，称为**尽力而为服务**（best-effort service）。从表 4-1 看，尽力而为服务看起来是根本无服务的一种委婉说法。使用尽力而为服务，分组间的定时是不能确保被保证的，分组接收的顺序也不能保证与发送的顺序一致，传送的分组也不能保证最终交付。给出这样的定义，一个没有向目的地交付分组的网络也符合尽力而为交付服务的定义。然而，如我们很快要讨论的那样，这样一种极为简化的网络层服务模型，是有其正当的存在理由的。

表 4-1 因特网、ATM CBR 和 ATM ABR 服务模型

网络体系结构	服务模型	带宽保证	无丢包保证	有序	定时	拥塞指示
因特网	尽力而为	无	无	任何可能的顺序	不维护	无
ATM	CBR	保证恒定速率	是	有序	维护	不出现拥塞
ATM	ABR	保证最小速率	无	有序	不维护	提供拥塞指示

其他的网络体系结构已定义和实现了许多超过因特网的尽力而为服务的服务模型。例如，ATM 网络体系结构 [MFA Forum 2012; Black 1995] 提供了多重服务模型，意味着可以在相同的网络中为不同的连接提供不同类别的服务。ATM 网络怎样提供这种服务的讨论已经超出了本书的范围；我们这里的目的是注意到除了因特网的尽力而为模型外，还存在其他模型即可。两个最重要的 ATM 服务模型是恒定比特率和可用比特率服务：

- **恒定比特率**（Constant Bit Rate, CBR）ATM 网络服务。这是第一个被标准化的 ATM 服务模型，它反映了电话公司对 ATM 的早期兴趣，以及 CBR 服务在承载实时、恒定比特率的音频和视频流量方面的适用性。CBR 服务的目标从概念上讲是简单的，就是使网络连接看起来就像在发送与接收主机之间存在一条专用的、固定带宽的传输链路，以使用性质相同的虚拟管道来提供分组（ATM 术语称为信元）流。使用 CBR 服务，ATM 信元流以如下方式被承载跨越网络，即一个信元的端到端时延、信元端到端时延中的可变性（即时延抖动）及丢失或推迟交付的信元的比率都确保在特定值以下。当第一次建立 CBR 连接时，发送主机与 ATM 网络认可了这些值。

- 可用比特率（Available Bit Rate, ABR）ATM 网络服务。由于因特网提供了所谓的尽力而为服务，ATM 的 ABR 也许最好被刻画为比尽力而为服务稍好一点的服务。与因特网服务模型一样，ABR 服务下的信元也许会丢失。然而与因特网不同的是，信元不能被重排序（虽然它们可能丢失），对于使用 ABR 服务的连接来说，最小信元传输速率（MCR）是可以得到保证的。如果在给定时间内网络有足够的空闲资源，发送方也可以用比 MCR 更高的速率成功地发送数据。另外，如我们在 3.6 节中所见，ATM ABR 服务能够为发送方提供反馈信息（利用一个拥塞通知比特，或一个明确的速率发送），以便控制发送方在 MCR 和一个允许的峰值信元速率之间调整其速率。

4.2 虚电路和数据报网络

回想第 3 章，运输层能够为应用程序提供无连接服务或面向连接服务。例如，因特网的运输层为每个应用程序在两种服务间提供了选择：UDP，一种无连接服务；或 TCP，一种面向连接服务。以类似的方式，网络层也能够在这两台主机之间提供无连接服务或连接服务。网络层的连接和无连接服务在许多方面与运输层的面向连接和无连接服务类似。例如，网络层连接服务以源和目的主机间的握手开始；网络层无连接服务则没有任何握手预备步骤。

尽管网络层连接和无连接服务与运输层面向连接和无连接服务有类似之处，但也存在重大差异：

- 在网络层中，这些服务是由网络层向运输层提供的主机到主机的服务。在运输层中，这些服务则是运输层向应用层提供的进程到进程的服务。
- 在至今为止的所有主要的计算机网络体系结构中（因特网、ATM、帧中继等），网络层或者提供了主机到主机的无连接服务，或者提供了主机到主机的连接服务，而不同时提供这两种服务。仅在网络层提供连接服务的计算机网络称为虚电路（Virtual-Circuit, VC）网络；仅在网络层提供无连接服务的计算机网络称为数据报网络（datagram network）。
- 在运输层实现面向连接的服务与在网络层实现连接服务是根本不同的。我们在前面一章看到，运输层面向连接服务是在位于网络边缘的端系统中实现的；我们很快看到，网络层连接服务除了在端系统中，也在位于网络核心的路由器中实现。

虚电路网络和数据报网络是计算机网络的两种基本类型。在作出转发决定时，它们使用了非常不同的信息。我们现在仔细观察一下它们的实现。

4.2.1 虚电路网络

虽然因特网是一个数据报网络，但许多其他网络体系结构（包括 ATM、帧中继的体系结构）却是虚电路网络，因此在网络层使用连接。这些网络层连接被称为虚电路。我们现在考虑在计算机网络中是如何实现虚电路服务的。

一条虚电路的组成如下：①源和目的主机之间的路径（即一系列链路和路由器）；②VC 号，沿着该路径的每段链路的一个号码；③沿着该路径的每台路由器中的转发表表项。属于一条虚电路的分组将在它的首部携带一个 VC 号。因为一条虚电路在每条链路上可能具有不同的 VC 号，每台中间路由器必须用一个新的 VC 号替代每个传输分组的 VC

号。该新的 VC 号从转发表获得。

为了举例说明这个概念，考虑在图 4-3 中的网络。在图 4-3 中靠近 R1 链路的号码是该链路接口的编号。现在假定主机 A 请求该网络在它自己与主机 B 之间创建一条虚电路。同时假定该网络为该虚电路选择路径 A-R1-R2-B 并为这条路径上的这 3 条链路分配 VC 号 12、22 和 32。在这种情况下，当在这条虚电路中的分组离开主机 A 时，在该分组首部中的 VC 字段的值是 12；当它离开 R1 时，该值是 22，而当它离开 R2 时，该值是 32。

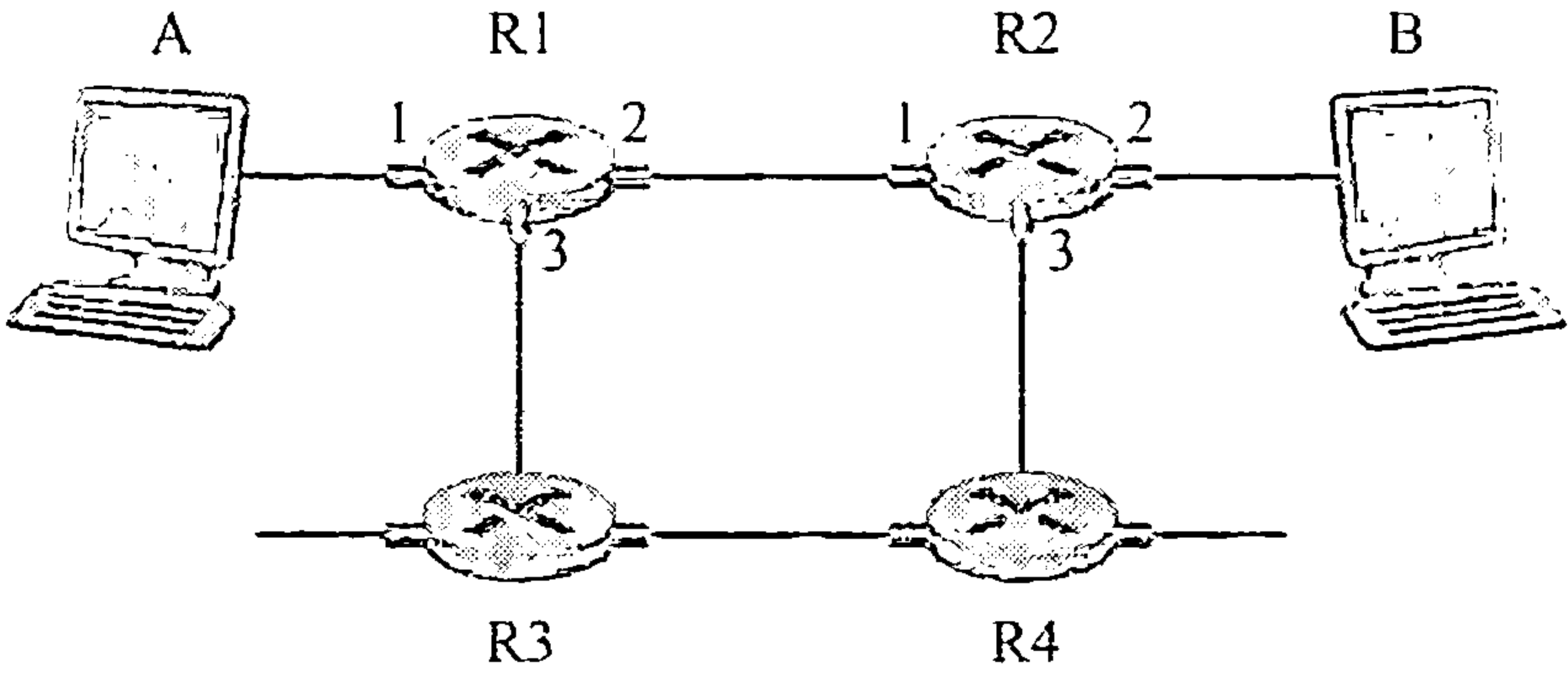


图 4-3 一个简单的虚电路网络

对于通过某路由器的分组，该路由器怎样决定更换其 VC 号呢？对于虚电路网络，每台路由器的转发表包括了 VC 号的转换；例如，在 R1 中的转发表可能有些像下表：

入接口	入 VC 号	出接口	出 VC 号
1	12	2	22
2	63	1	18
3	7	2	17
4	97	3	87
...	...	...	...

无论何时跨越一台路由器创建一条新的虚电路，转发表就增加了一个新表项。类似地，无论何时终止一条虚电路，沿着该路径每个表中的相应项将被删除。

你也许想知道一个分组沿着其路由在每条链路上不简单地保持相同的 VC 号的原因。答案包括两方面。第一，逐链路代替该号码减少了在分组首部中 VC 字段的长度。第二方面更为重要，通过允许沿着该虚电路路径每条链路有一个不同的 VC 号，大大简化了虚电路的建立。特别是，在具有多个 VC 号的路径，其上的每条链路能够独立于沿着该路径的其他链路所选的号码选择一个 VC 号。如果沿着某路径的所有链路要求一个共同的 VC 号的话，路由器将不得不交换并处理相当大量的报文，以约定一个共同的 VC 号（例如，一个并未由这些路由器的任何其他现有虚电路使用的号码）用于一次连接。

在虚电路网络中，该网络的路由器必须为进行中的连接维持连接状态信息（connection state information）。特别是，每当跨越一台路由器创建一个新连接，必须在该路由器的转发表中增加一个新的连接项；每当释放一个连接，必须从该表中删除该项。值得注意的是，即使没有 VC 号转换，仍有必要维持连接状态信息，该信息将 VC 号与输出接口号联系起来。路由器是否对每条进行中的连接维持连接状态信息是一个关键性问题，我们将在本书中将再次讨论该问题。

在虚电路中有 3 个明显不同的阶段：

- 虚电路建立。在建立阶段，发送运输层与网络层联系，指定接收方地址，等待网络建立虚电路。网络层决定发送方与接收方之间的路径，即该虚电路的所有分组要通过的一系列链路与路由器。网络层也为沿着该路径的每条链路决定一个 VC 号。最后，网络层在沿着路径的每台路由器的转发表中增加一个表项。在虚电路建立期间，网络层还可以预留该虚电路路径上的资源（如带宽）。

- 数据传送。如图 4-4 中所示，一旦创建了虚电路，分组就可以开始沿该虚电路流动了。

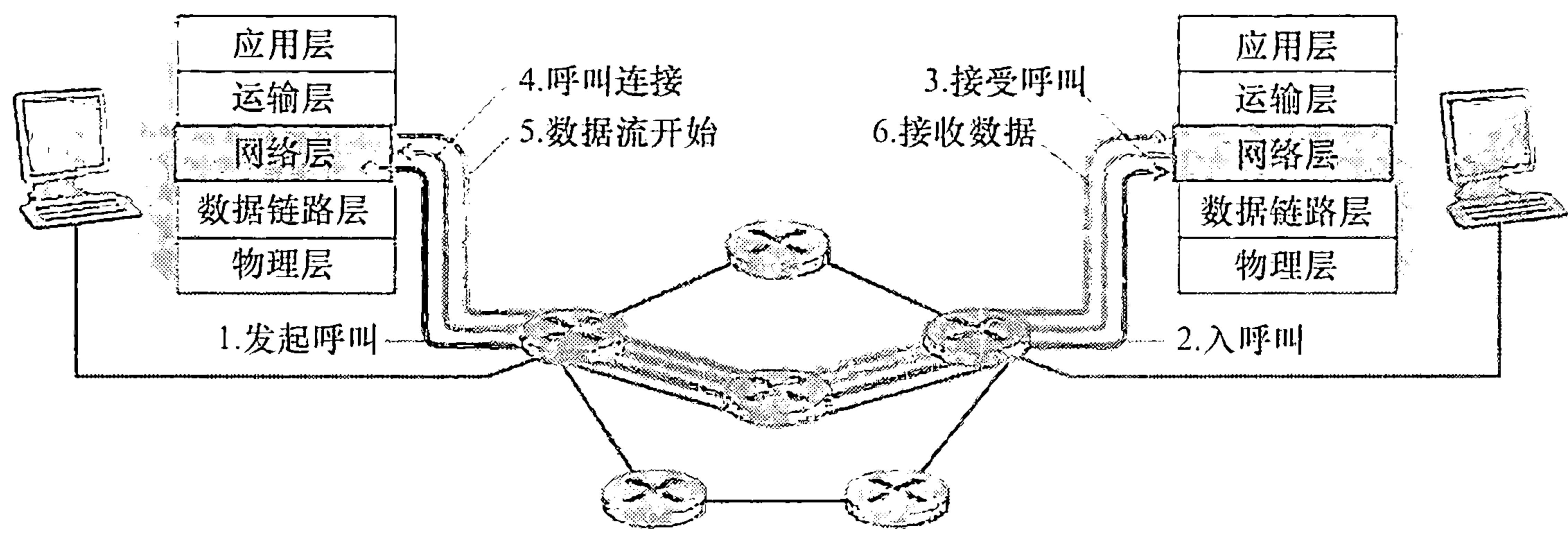


图 4-4 虚电路建立

- 虚电路拆除。当发送方（或接收方）通知网络层它希望终止该虚电路时，就启动这个阶段。然后网络层通常将通知网络另一侧的端系统结束呼叫，并更新路径上每台分组路由器中的转发表以表明该虚电路已不存在了。

在网络层的虚电路建立与运输层的连接建立（例如我们在第 3 章学习的 TCP 三次握手）之间有一个细微但很重要的区别。运输层的连接建立仅涉及两个端系统。在运输层的连接建立期间，两个端系统独自决定运输层连接参数（如初始序号与流量控制窗口长度）。虽然这两个端系统已经知道该运输层连接，但网络中的路由器则对这些完全不知情。在另一方面，对于一个虚电路网络层，沿两个端系统之间路径上的路由器都要参与虚电路的建立，且每台路由器都完全知道经过它的所有虚电路。

端系统向网络发送指示虚电路启动与终止的报文，以及路由器之间传递的用于建立虚电路（即修改路由器表中的连接状态）的报文，它们被称为信令报文（signaling message），用来交换这些报文的协议常称为信令协议（signaling protocol）。虚电路建立如图 4-4 所示。在本书中我们将不涉及虚电路信令协议；有关面向连接网络中信令的一般讨论参见 [Black 1997]，有关 ATM 的 Q. 2931 信令协议的规范参见 [ITU-T Q. 2931 1995]。

4.2.2 数据报网络

在数据报网络中，每当一个端系统要发送分组，它就为该分组加上目的端系统的地址，然后将分组推进网络中。如图 4-5 中所示，无需建立任何虚电路，路由器不维护任何虚电路的状态信息（因为没有虚电路！）。

当分组从源到目的地传输，它通过一系列路由器传递。这些路由器中的每台都使用分组的目的地址来转发该分组。特别是，每台路由器有一个将目的地址映射到链路接口的转发表；当分组到达路由器时，路由器使用该分组的目的地址在转发表中查找适当的输出链路接口。然后路由器有意将分组向该输出链路接口转发。

为了进一步深入理解查找操作，我们看一个特定的例子。假定所有的目的地地址均是 32 比特（这恰好就是在 IP 数据报中目的地址的长度）。转发表的蛮力实现将对每个可能的目的地址有一个表项。因为有超过 40 亿个可能的地址，这种选择完全是不可能的。

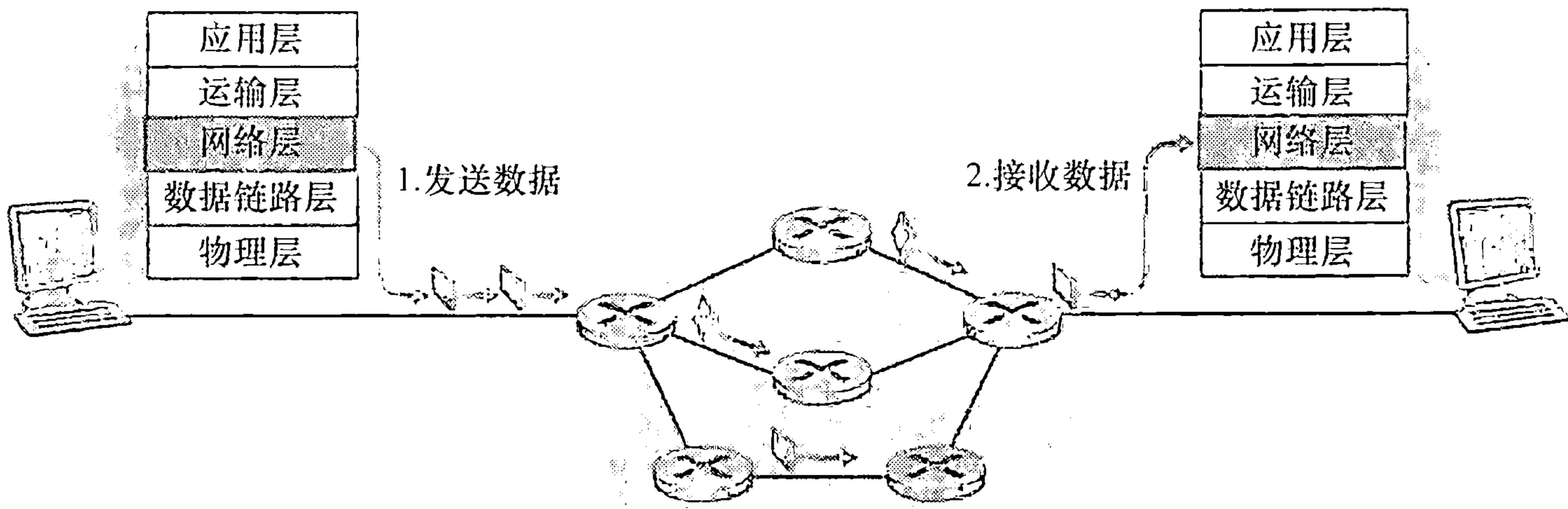


图 4-5 数据报网络

现在我们进一步假设路由器有 4 条链路，编号 0 ~ 3，分组以如下方式转发到链路接口：

目的地址范	链路接口
11001000 00010111 00010000 00000000 到 11001000 00010111 00010111 11111111	0
11001000 00010111 00011000 00000000 到 11001000 00010111 00011000 11111111	1
11001000 00010111 00011001 00000000 到 11001000 00010111 00011111 11111111	2
其他	3

显然，对于这个例子，在路由器的转发表中就没有必要有 40 亿表项。例如，我们能够有仅包括 4 个表项的下列转发表：

前缀匹配	链路接口
11001000 00010111 00010	0
11001000 00010111 00011000	1
11001000 00010111 00011	2
其他	3

使用这种风格的转发表，路由器用分组的目的地址的前缀（prefix）与该表中的表项进行匹配；如果存在一个匹配项，则路由器向与该匹配项相联系的链路转发分组。例如，假设分组的目的地址是 11001000 00010111 00010110 10100001；因为该地址的 21 比特前缀匹配该表的第一项，所以路由器向链路接口 0 转发该分组。如果一个前缀不匹配前 3 项中的任何一项，则路由器向链路接口 3 转发该分组。尽管听起来足够简单，但这里还是有重要的微妙之处。你可能已经注意到一个目的地址可能与不止一个表项相匹配。例如，地址 11001000 00010111 00010000 10101010 的前 24 比特与表中的第二项匹配，而该地址的前 21 比特与表中的第三项匹配。当有多个匹配时，该路由器使用最长前缀匹配规则（longest prefix matching rule）；即在该表中寻找最长的匹配项，并向与最长前缀匹配相关联的链路接口转发分组。当我们在 4.4 节中详细学习因特网编址时，我们将完全明白使用这种最长

前缀匹配规则的理由。

虽然在数据报网络中的路由器不维持连接状态信息，但它们无论如何在其转发表中维持了转发状态信息。然而，转发状态信息表变化的时间尺度相对要慢。实际上，在数据报网络中的转发表是通过路由选择算法进行修改的，这通常每 1 ~ 5 分钟左右更新一次转发表。在虚电路网络中，无论何时通过路由器建立一条新的连接，或无论何时通过路由器拆除一条现有的连接，路由器中的转发表就被更新。对一台第一层主干路由器而言，这很容易以微秒的时间尺度进行更新。

因为在数据报网络中的转发表能够在任何时刻修改，从一个端系统到另一个端系统发送一系列分组可能在通过网络时走不同的路径，并可能无序到达。[Paxson 1997] 和 [Jaiswal 2003] 对公共因特网上的分组重排序和其他现象进行了有趣的测量研究。

4.2.3 虚电路和数据报网络的由来

数据报与虚电路网络的演化反映了它们的由来。作为一条重要的组织原则，虚电路的概念来源于电话界，它采用了真正的电路。由于呼叫建立及每呼叫的状态要在网络中的路由器上维持，一个面向虚电路的网络显然比数据报网络要复杂得多（对于电路交换与分组交换网络的复杂性的有趣比较可参看 [Molinero-Fernandez 2002]）。这也与它的电话传统一致。电话网络在网络中必然有其复杂性，因为它们要连接哑端系统设备，如转盘电话。（对于很年轻的人来说可能不知道转盘电话为何物，这是一种无按键、仅有一个拨号盘的模拟电话。）

另一方面，因特网作为一种数据报网络，是由将计算机连接在一起的需求发展而来的。由于端系统设备复杂得多，因特网架构师们选择使网络层服务模型尽可能简单。如我们在第 2、3 章中所见，另外的功能（例如，按序传送、可靠数据传输、拥塞控制与 DNS 名字解析）在端系统中的更高层实现。这正好与电话网模型相反，并产生了一些有趣的结果。

- 由于所产生的因特网服务模型使服务保证最少（没有！），它对网络层施加了最小限度的需求。这使得互联使用各种不同链路层技术而且具有十分不同的传输速率和丢包特性的网络变得更加容易。这些链路层技术包括卫星、以太网、光纤或无线。我们将在 4.4 节中详细地讨论 IP 网络的互联。
- 如我们在第 2 章所见，诸如电子邮件、Web 等应用，甚至如 DNS 这样的网络基础设施都是在位于网络边缘的主机（服务器）上实现的。增加一个新服务只需连接一台主机到网络中，并定义一个新的应用层协议（如 HTTP）即可，这种能力可以使如 Web 之类的新服务能在相当短的时间内在因特网上得以部署。

4.3 路由器工作原理

既然我们已经概述了网络层的功能和服务，我们将注意力转向网络层的转发功能（forwarding function），即实际将分组从一台路由器的人链路传送到适当的出链路。我们已经在 4.2 节中简要地了解了一些转发问题，也就是编址和最长前缀匹配问题。我们以前提到过，计算机网络研究人员和从业人员经常互换使用转发和交换这两个词；在本书中我们也将交互使用这两个词。

图 4-6 显示了一个通用路由器体系结构的总体视图。其中标识了一台路由器的 4 个组成部分。

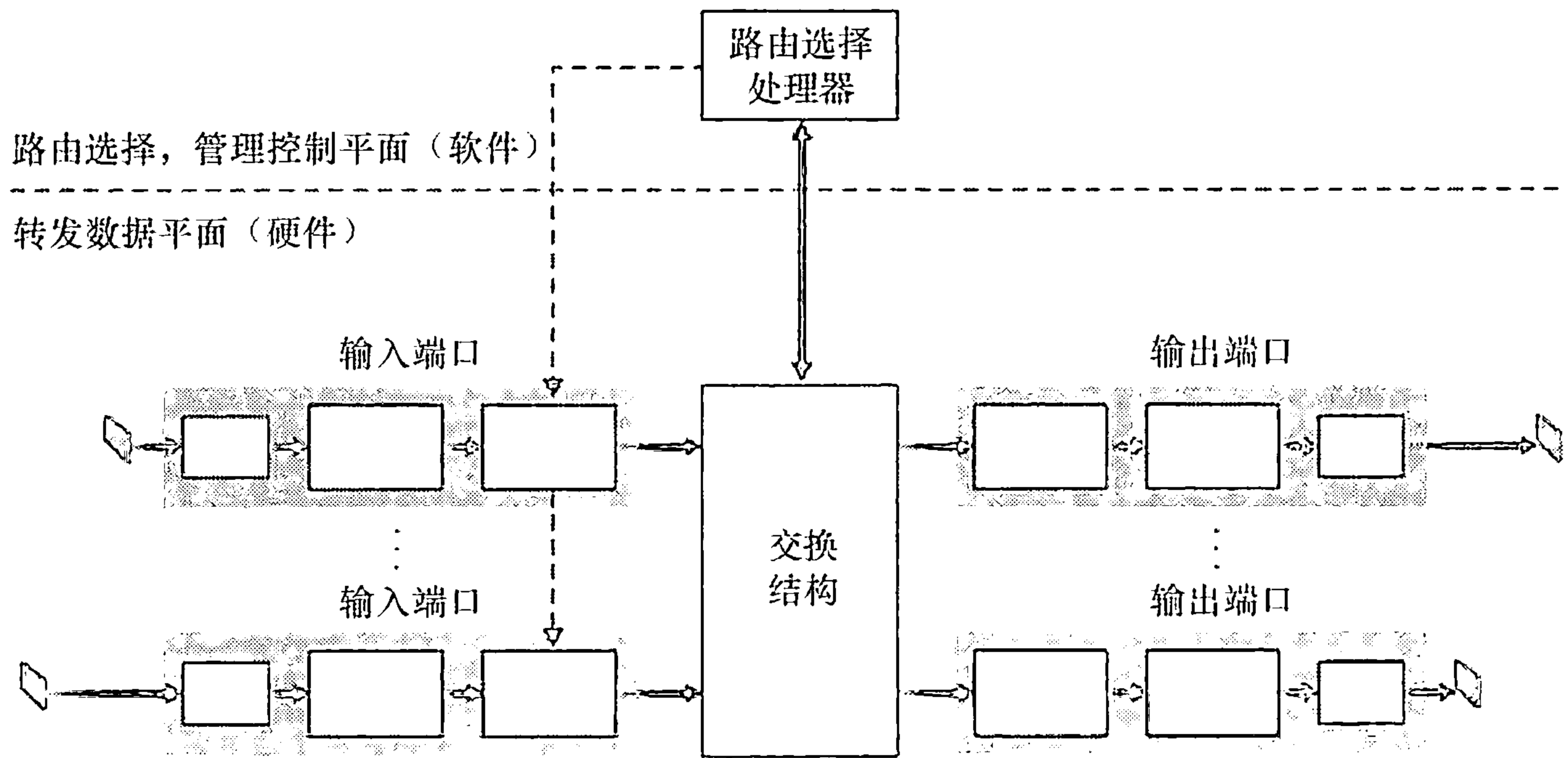


图 4-6 路由器体系结构

- 输入端口。输入端口执行几项关键功能。它要执行将一条输入的物理链路 with 路由器相连接的物理层功能，这显示在图 4-6 中输入端口部分最左侧的方框与输出端口部分最右侧的方框中。它还要执行需要与位于入链路远端的数据链路层交互的数据链路层功能，这表示在输入与输出端口部分的中间方框中。也许更为重要的是，在输入端口还要完成查找功能，这显示在输入端口最右侧的方框中。正是在这里，通过查询转发表决定路由器的输出端口，到达的分组通过路由器的交换结构将转发到输出端口。控制分组（如携带路由选择协议信息的分组）从输入端口转发到路由选择处理器。注意这里的端口一词，是指路由器的物理输入和输出接口，这完全不同于第 2、3 章中与网络应用程序和套接字相联系软件端口。
- 交换结构。交换结构将路由器的输入端口与输出端口相连接。这种交换结构完全包含在路由器中，即它是一个网络路由器中的网络！
- 输出端口。输出端口存储从交换结构接收的分组，并通过执行必要的链路层和物理层功能在输入链路上传输这些分组。当一条链路是双向的（即承载两个方向的流量）时，输出端口通常是与该链路的输入端口在同一线路卡（一个包含一个或多个输入端口的印刷电路，它与交换结构相连）上成对出现的。
- 路由选择处理器。路由选择处理器执行路由选择协议（我们将在 4.6 节中学习），维护路由选择表以及连接的链路状态信息，并为路由器计算转发表。它还执行网络管理功能，我们将在第 9 章学习相关内容。

4.1.1 节讲过，我们对路由器的转发功能和路由选择功能加以区分。一台路由器的输入端口、输出端口和交换结构共同实现了这种转发功能，并且总是用硬件实现，如图 4-6 所示。这些转发功能有时总称为路由器转发平面（router forwarding plane）。为了理解为何需要硬件实现，考虑具有 10Gbps 带宽的输入链路和 64 字节的 IP 数据报，其输入端口在另一个数据报到达前仅有 51.2ns 来处理数据报。如果 N 个端口结合在一块线路卡上（因为实践中常常这样做），数据报处理流水线必须以 N 倍速率运行，这远快过软件实现的速率。

转发平面硬件既能够使用厂商自己的硬件设计来实现，也能够使用购买的商用硅芯片（例如英特尔和 Broadcom 公司所出售的）来实现。

当转发平面以纳秒时间尺度运行时，路由器的控制功能（即执行路由选择协议、对上线或下线的连接链路进行响应，以及执行如我们将在第 9 章中学习的管理功能），在毫秒或秒时间尺度上运行。这些路由器控制平面（router control plane）通常用软件实现并在路由选择处理器上执行（通常是一种传统的 CPU）。

在深入探讨路由器的控制和数据平面的细节之前，我们转向 4.1.1 节中的类比，其中分组转发好比汽车进入和离开立交桥。假定立交桥是环状交叉路，在汽车进入该环状交叉路前，需要做一点处理，即汽车停在一个入口站上并指示它的最终目的地（并非在本地环状交叉路，而是其旅途的最终目的地）。入口站的一名服务人员查找最终目的地，决定通向最后目的地的环状交叉路的出口，并告诉驾驶员要走哪个出口。该汽车进入环状交叉路（该环状交叉路可能挤满了从其他输入道路进入的其他汽车，朝着其他环状交叉路出口前进），并且最终离开预定的环状交叉路出口斜道，在这里可能遇到了在该出口离开环状交叉路的其他汽车。

在这个类比中，我们能够识别图 4-6 中的最重要的路由器组件，即入口路和入口站对应于输入端口（具有查找功能以决定本地输出端口）；环状交叉路对应于交换结构；环状交叉路出口路对应于输出端口。借助于这个类比，考虑瓶颈可能出现的地方是有益的。如果汽车以极快的速率到达（例如，该环状交叉路位于德国或意大利！）而车站服务人员很慢，将发生什么情况？这些服务人员必须工作得多快，以确保在入口路上没有车辆拥堵？甚至对于极快的服务人员，如果汽车在环状交叉路上开得很慢，将发生什么情况？拥堵仍会出现吗？如果大多数进入的汽车都要在相同的出口斜道离开环状交叉路，将发生什么情况？在出口斜道或别的什么地方会出现拥堵吗？如果我们要为不同的汽车分配优先权，或先行阻挡某些汽车进入环状交叉路，环状交叉路将如何运行？这些全都与路由器和交换机设计者面对的问题形成类比。

在下面的各小节中，我们将更为详细地考察路由器功能。[Iyer 2008；Chao 2001；Chuang 2005；Turner 1988；McKeown 1997a；Partridge 1998] 提供了对一些特定路由器体系结构的讨论。为了具体起见，后继的讨论假定一个数据报网络中转发决定是基于分组的目的地地址（而非虚电路网络中的 VC 号）的。然而，对于虚电路网络而言，概念和技术是相当类似的。

历史事件

Cisco 系统：主宰网络的核心

在 2012 年写本书时，Cisco 公司雇佣了 65 000 多人。这个网络公司的巨人是如何发展起来的呢？它的一切是从 1984 年在硅谷公寓的一间起居室里起步的。

当 Len Bosak 与他的妻子 Sandy Lerner 在斯坦福大学工作时，他们就有建造并出售因特网路由器给研究及学术团体的想法，他们成为那个时期因特网的主要培育者。Sandy Lerner 起了一个名字 Cisco（这是旧金山的一种略称），还设计了公司的桥形标志图案。公司总部最初是他们的起居室，而且刚开始时他们通过信用卡和夜间兼职咨询工

作为该项目筹措资金。到了1986年年末，Cisco的月收入达到了250 000美元。到1987年年末，Cisco以1/3公司股份为交换条件，终于成功吸引到了来自Sequoia Capital的200万美元的风险投资。在后来的几年中，Cisco持续增长并抢占了越来越多的市场份额。与此同时，Bosak/Lerner与Cisco管理层之间的关系变得紧张起来。Cisco在1990年公开发售股票，但在同年，Lerner与Bosak离开了公司。

多年以来，Cisco在路由器市场之外进行了成功的扩展，销售安全、无线缓存、以太网交换机、数据中心基础设施、视频会议和IP语音产品和服务。然而，Cisco正面临着日益增长的国际竞争，包括来自华为（一个迅速增长的中国网络设备公司）的竞争。在路由器和交换以太网领域与Cisco竞争的公司包括Alcatel-Lucent和Juniper。

4.3.1 输入端口

图4-7中给出一个更详细的输入处理的视图。如前面讨论的那样，输入端口的线路端接功能与链路层处理实现了用于各个输入链路的物理层和链路层。在输入端口中执行的查找对于路由器的运行是至关重要的。正是在这个地方，路由器使用转发表来查找输出端口，使得到达的分组将能经过交换结构转发到该输出端口。转发表是由路由选择处理器计算和更新的，但转发表的一份影子副本通常会被存放在每个输入端口。转发表从路由选择处理器经过独立总线（例如一个PCI总线）复制到线路卡，在图4-6中该总线由从路由选择处理器到输入线路卡的虚线所指示。有了影子副本，转发决策能在每个输入端口本地做出，无须调用中央路由选择处理器，因此避免了集中式处理的瓶颈。

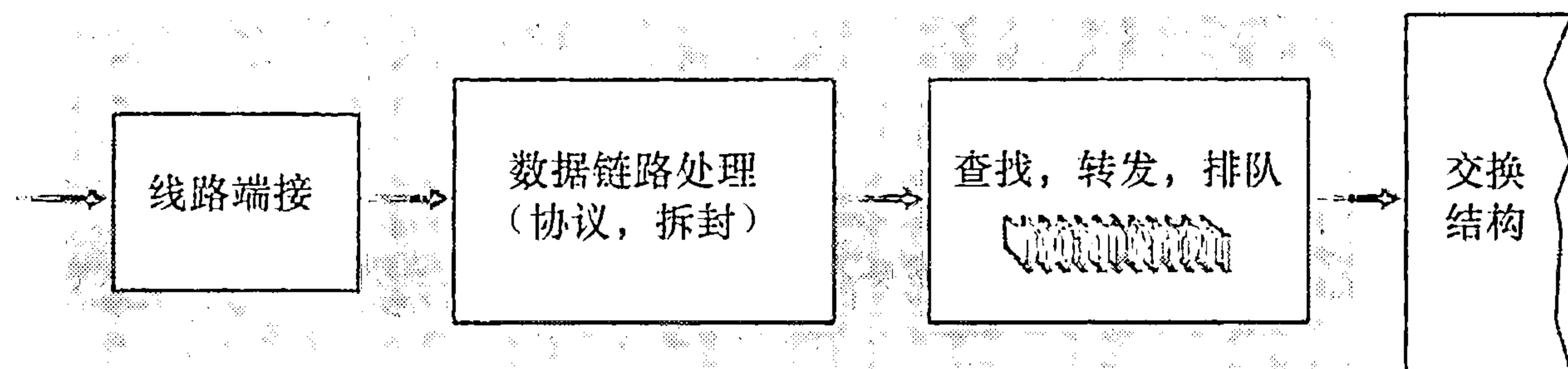


图 4-7 输入端口处理

假定转发表已经存在，从概念上讲表查找是简单的，即我们只是搜索转发表查找最长前缀匹配，如4.2.2节所描述的那样。但在吉比特速率下，这种查找必须在纳秒级执行（回想我们前面10Gbps链路和一个64字节IP数据报）。因此，不仅必须要用硬件执行查找，而且需要对大型转发表使用超出简单线性搜索的技术；快速查找算法的综述能够在[Gupta 2001, Ruiz-Sanchez 2011]中找到。同时必须对内存访问时间给予特别关注，这导致用嵌入式片上DRAM和更快的SRAM（用作一种DRAM缓存）内存来设计。三态内容可寻址存储器（Ternary Content Address Memory, TCAM）也经常被用于查找。使用一个TCAM，一个32比特IP地址被放入内存，TCAM在基本常数时间内返回对该地址的转发表项的内容。Cisco 8500对每个输入端口有一个64K的CAM。

一旦通过查找确定了某分组的输出端口，则该分组就能够发送进入交换结构。在某些设计中，如果来自其他输入端口的分组当前正在使用该交换结构，一个分组可能会在进入交换结构时被暂时阻塞。因此，一个被阻塞的分组必须要在输入端口处排队，并等

待稍后被及时调度以通过交换结构。我们将在 4.3.4 节中仔细观察分组（位于输入与输出端口中）的阻塞、排队与调度。尽管“查找”在输入端口处理中可以说是最为重要的动作，但必须采取许多其他动作：①必须出现物理层和链路层处理，如前面所讨论的那样；②必须检查分组的版本号、检验和以及寿命字段（这些我们将在 4.4.1 节中学习），并且重写后两个字段；③必须更新用于网络管理的计数器（如接收到的 IP 数据报的数目）。

在结束输入端口处理的讨论之前，我们注意到输入端口查找 IP 地址然后发送该分组进入交换结构（“动作”）的步骤是一种更为一般的“匹配加动作”抽象的特定情况，这种抽象执行在许多网络设备中，而不仅在路由器中。在链路层交换机（在第 5 章讨论）中，除了发送帧进入交换结构去往输出端口外，还要查找链路层目的地址，并采取几个动作。在防火墙（包含在第 8 章）中，首部匹配给定准则（例如源/目的 IP 地址和运输层端口号的某种组合）的入分组可能被阻止转发，而防火墙是一种过滤所选择的入分组的设备。在网络地址转换（NAT，包含在 4.4 节中）中，一个运输层端口号匹配某给定值的入分组，在转发（动作）前其端口号将被重写。因此，“匹配加动作”抽象不仅作用大，而且在网络设备中无所不在。

4.3.2 交换结构

交换结构位于一台路由器的核心部位。正是通过这种交换结构，分组才能实际地从一个输入端口交换（即转发）到一个输出端口中。交换可以用许多方式完成，如图 4-8 所示。

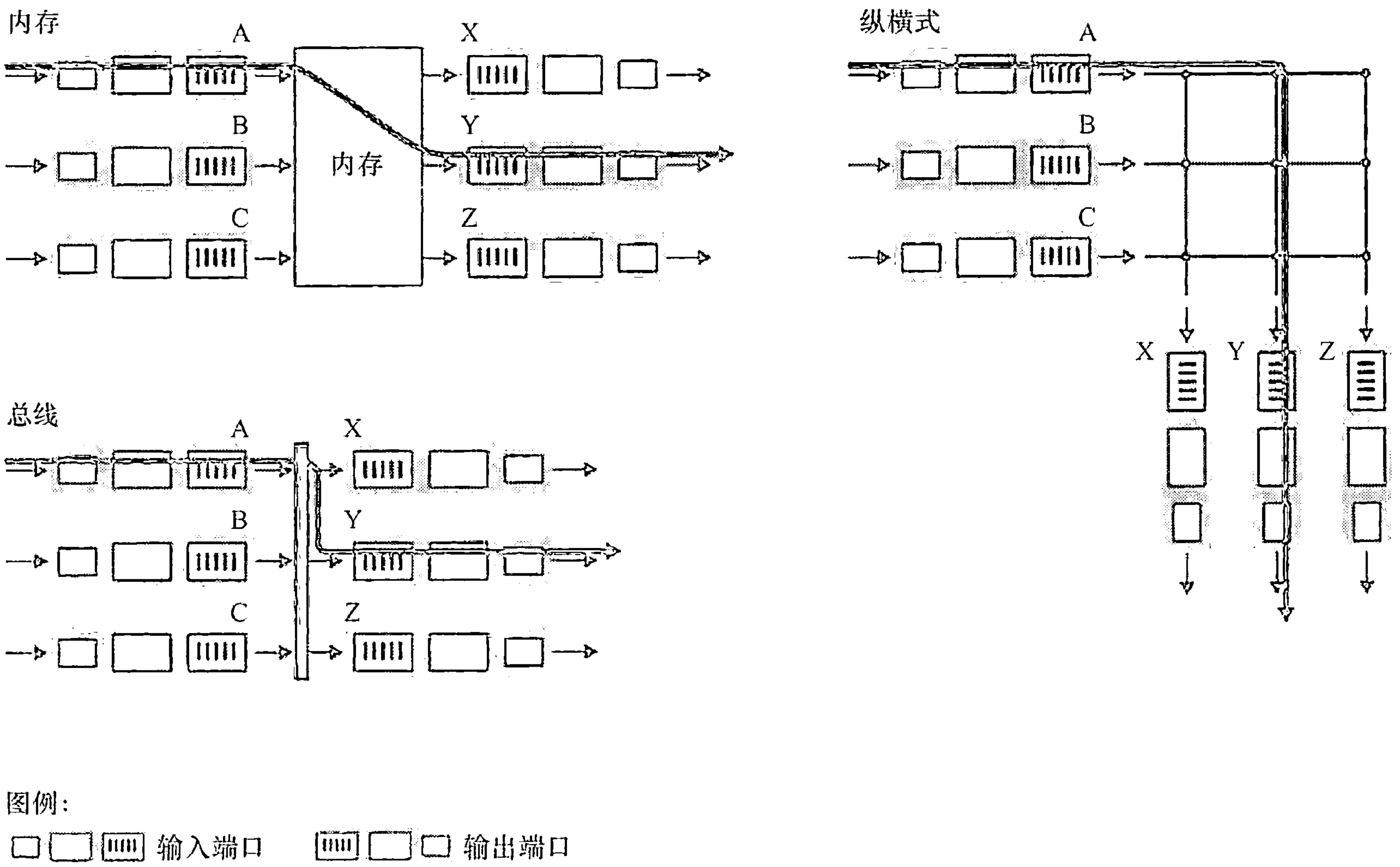


图 4-8 三种交换技术

- 经内存交换。最简单、最早的路由器是传统的计算机，在输入端口与输出端口之间的交换是在 CPU（路由选择处理器）的直接控制下完成的。输入与输出端口的功能就像在传统操作系统中的 I/O 设备一样。一个分组到达一个输入端口时，该端口会先通过中断方式向路由选择处理器发出信号。于是，该分组从输入端口处被复制到处理器内存中。路由选择处理器则从其首部中提取目的地址，在转发表中找出适当的输出端口，并将该分组复制到输出端口的缓存中。在这种情况下，如果内存带宽为每秒可写进内存或从内存读出 B 个分组，则总的转发吞吐量（分组从输入端口被传送到输出端口的总速率）必然小于 $B/2$ 。也要注意不能同时转发两个分组，即使它们有不同的端口号，因为经过共享系统总线一次仅能执行一个内存读/写。

许多现代路由器通过内存进行交换。然而，与早期路由器的一个主要差别是，目的地址的查找和将分组存储（交换）进适当的内存存储位置是由输入线路卡来处理的。在某些方面，经内存交换的路由器看起来很像共享内存的多处理机，用一个线路卡上的处理将分组交换（写）进适当的输出端口的内存中。Cisco 的 Catalyst 8500 系列的交换机 [Cisco 8500 2012] 是经共享内存转发分组的。

- 经总线交换。在这种方法中，输入端口经一根共享总线将分组直接传送到输出端口，不需要路由选择处理器的干预。通常按以下方式完成该任务：让输入端口为分组预先计划一个交换机内部标签（首部），指示本地输出端口，使分组在总线上传送和传输到输出端口。该分组能由所有输出端口收到，但只有与该标签匹配的端口才能保存该分组。然后标签在输出端口被去除，因为其仅用于交换机内部来跨越总线。如果多个分组同时到达路由器，每个位于不同的输出端口，除了一个分组外所有其他分组必须等待，因为一次只有一个分组能够跨越总线。因为每个分组必须跨过单一总线，故路由器的交换带宽受总线速率的限制；在我们环状交叉路的类比中，这相当于环状交叉路一次仅包含一辆汽车。尽管如此，对于运行在小型局域网和企业网中的路由器来说，通过总线交换通常是足够的。Cisco 5600 [Cisco Switches 2012] 交换机通过一个 32Gbps 背板总线来交换分组。
- 经互连网络交换。克服单一、共享式总线带宽限制的一种方法是，使用一个更复杂的互连网络，例如过去在多处理器计算机体系结构中用来互联多个处理器的网络。纵横式交换机就是一种由 $2N$ 条总线组成的互连网络，它连接 N 个输入端口与 N 个输出端口，如图 4-8 所示。每条垂直的总线在交叉点与每条水平的总线交叉，交叉点通过交换结构控制器（其逻辑是交换结构自身的一部分）能够在任何时候开启和闭合。当某分组到达端口 A，需要转发到端口 Y，交换机控制器闭合总线 A 和 Y 交叉部位的交叉点，然后端口 A 在其总线上发送该分组，该分组仅由总线 Y 安排接收。注意到来自端口 B 的一个分组在同一时间能够转发到端口 X，因为 A 到 Y 和 B 到 X 的分组使用不同的输入和输出总线。因此，与前面两种交换方法不同，纵横式网络能够并行转发多个分组。然而，如果来自两个不同输入端口的两个分组其目的地为相同的输出端口，则一个分组必须在输入端等待，因为在某个时刻经给定总线仅有一个分组能够发送。

更为复杂的互联网络使用多级交换元素，以使来自不同输入端口的分组通过交换结构同时朝着相同的输出端口前行。对交换机体系结构的展望可参见 [Tobagi 1990]。Cisco 12000 系列交换机 [Cisco 12000 2012] 使用了一个互联网络。

4.3.3 输出端口

如图 4-9 中所示，输出端口处理取出存放在输出端口内存中的分组并将其发送到输出链路上。这包括选择和取出排队的分组进行传输，执行所需的链路层和物理层传输功能。

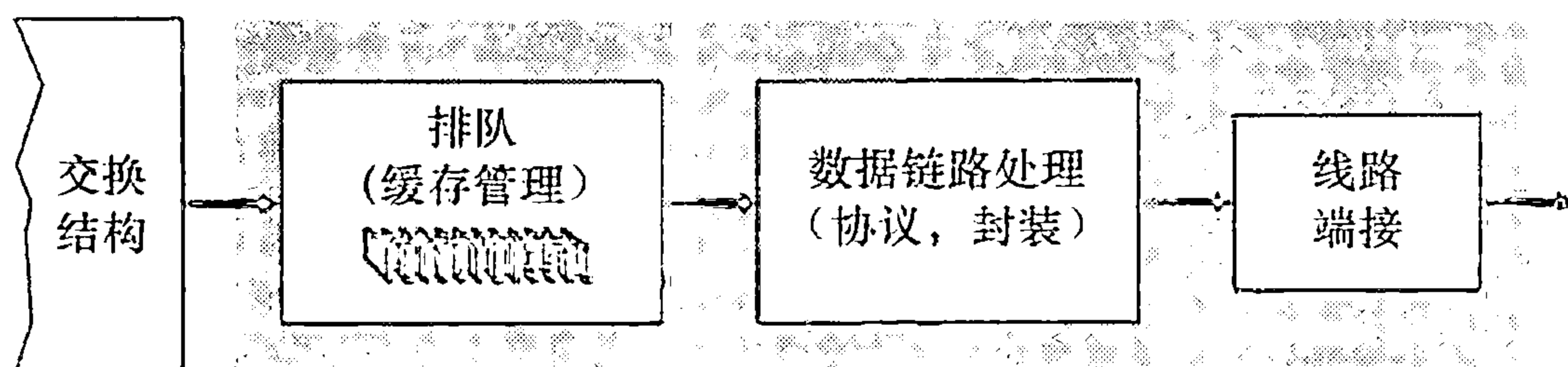


图 4-9 输出端口处理

4.3.4 何处出现排队

如果我们考虑显示在图 4-8 中的输入和输出端口功能及其配置，就会明显发现在输入端口和输出端口处都能够形成分组队列，就像在环状交叉路的类比中我们已知的情况，汽车可能等待在流量交叉点的入口和出口。排队的位置和程度（或者在输入端口排队，或者在输出端口排队）将取决于流量负载、交换结构的相对速率和线路速率。我们现在更为详细一点考虑这些队列，因为随着这些队列的增长，路由器的缓存空间将最终会耗尽，并且当无内存可用于存储到达的分组时将会出现丢包（packet loss）。回想前面的讨论，我们说过分组“在网络中丢失”或“被路由器丢弃”。正是在一台路由器的这些队列中，这些分组被实际丢弃或丢失。

假定输入线路速度与输出线路速度是相同的，均为每秒 R_{line} 个分组，有 N 个输入端口和 N 个输出端口。为进一步简化讨论，假设所有分组具有相同的固定长度，分组以同步的方式到达输入端口。这就是说，在任何链路发送分组的时间等于在任何链路接收分组的时间，在这样的时间间隔内，在一个输入链路上能够到达 0 个或 1 个分组。定义交换结构传送速率 R_{switch} 为从输入端口到输出端口能够移动分组的速率。如果 R_{switch} 比 R_{line} 快 N 倍，则在输入端口处仅会出现微不足道的排队。这是因为即使在最坏情况下，所有 N 条输入线路都在接收分组，并且所有的分组将被转发到相同的输出端口，每批 N 个分组（每个输入端口一个分组）能够在下一批到达前通过交换结构处理完毕。

但是在输出端口处会发生什么情况呢？我们仍然假设 R_{switch} 比 R_{line} 快 N 倍。同样，到达 N 个输入端口的每个端口的分组都发向相同的输出端口。在这种情况下，在向输出链路发送一个分组的时间内，将有 N 个分组到达该输出端口。因为输出端口在一个单位时间（分组传输时间）内只能发送一个分组，这 N 个到达的分组必须排队（等待）传输到输出链路上。于是，又有 N 个分组可能在它只能发送已排队的 N 个分组中的一个的时间内到达。这种情况会不断持续下去。最终，排队的分组数量会增长得很快，足以耗尽输出端口的可用内存，在这样的情况下分组被丢弃。

在图 4-10 中图示了输出端口的排队情况。在时刻 t ，每个输入端口都到达了一个

分组，每个分组都是发往最上侧的输出端口。假定线路速度相同，交换以三倍快的线路速度进行操作，一个时间单位以后（即接收或发送一个分组所需的时间），所有三个初始分组都被传送到输出端口，并排队等待传输。在下一个时间单位中，这三个分组中的一个将通过输出链路发送出去。在这个例子中，又有两个新分组已到达交换机的入端；这些分组之一要发往最上侧的输出端口。

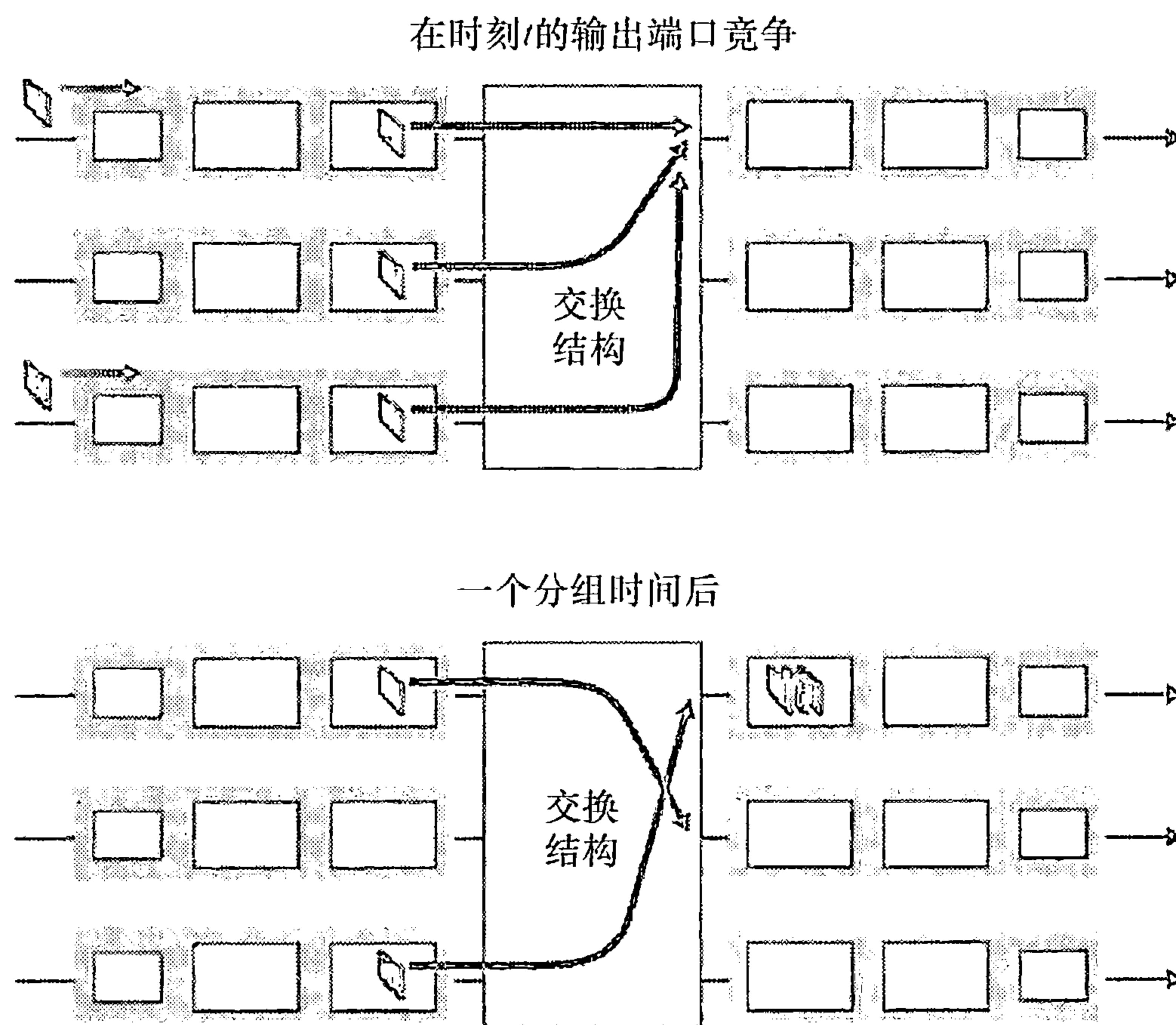


图 4-10 输出端口排队

假定需要路由器缓存来吸收流量负载的波动，一个自然而然的问题就是需要多少缓存。多年以来，对缓存长度的经验方法是 [RFC 3439]，缓存数量 (B) 应当等于平均往返时延 (RTT，比如说 250ms) 乘以链路的容量 (C)。这个结果是基于相对少量的 TCP 流的排队动态性分析得到的 [Villamizar 1994]。因此，一条具有 250ms RTT 的 10Gbps 链路将需要的缓存量等于 $B = \text{RTT} \cdot C = 2.5\text{Gb}$ 。然而，最近的理论和试验研究 [Appenzeller 2004] 表明，当有大量的 TCP 流 (N) 流过一条链路时，缓存所需要的数量是 $B = \text{RTT} \cdot C / \sqrt{N}$ 。对于通常有大量流经过的大型主干路由器链路（参见如 [Fraleigh 2003]）， N 的值可能非常大，这使得所需的缓存长度的减小相当明显。[Appenzeller 2004; Wischik 2005; Beheshti 2008] 从理论、实现和运行的角度提供了可读性很强的有关缓存长度问题的讨论。

输出端口排队的后果就是，在输出端口上的一个分组调度程序 (packet scheduler) 必须要在这些排队的分组中选出一个来发送。这种选择可能是根据简单的原则来定，如先来先服务 (FCFS) 调度，或者更复杂的调度规则，如加权公平排队 (WFQ)。WFQ 规则是在具有排队等待传输的分组的不同端到端连接之间公平地共享输出链路。分组调度程序在提供服务质量保证 (quality-of-service guarantee) 方面起着关键作用。我们将在第 7 章更深入地讨论分组调度。有关输出端口分组调度规则的讨论见 [Cisco Queue 2012]。

类似地，如果没有足够的内存来缓存一个入分组，那么必须做出决定：要么丢弃到达的分组（一种称为弃尾 (drop-tail) 的策略），要么删除一个或多个已排队的分组来为新

到的分组腾出空间。在某些情况下，在缓存填满前便丢弃（或在首部加标记）一个分组，以便向发送方提供一个拥塞信号的做法是有利的。已经提出和分析了许多分组丢弃与标记策略 [Labrador 1999, Hollot 2002]，这些策略统称为主动队列管理（Active Queue Management, AQM）算法。随机早期检测（Random Early Detection, RED）算法是一种得到最广泛研究和实现的 AQM 算法。在 RED 算法中，为输出队列长度维护着一个加权平均值。如果平均队列长度小于最小阈值 $\min_{th}$ ，则当一个分组到达时，该分组被接纳进队列。相反，如果队列满或平均队列长度大于最大阈值 $\max_{th}$ ，则当一个分组到达时，该分组被标记或丢弃。最后，如果一个分组到达，发现平均队列长度在 $[\min_{th}, \max_{th}]$ 之间时，则该分组以某种概率被标记或丢弃，该概率一般是平均队列长度、 $\min_{th}$ 和 $\max_{th}$ 的某种函数。已提出了一些概率标记/丢弃函数，各种版本的 RED 已被分析建模、模拟和/或实现。[Christiansen 2001] 与 [Floyd 2012] 为课外阅读提供了综述及指南。

如果交换结构不能快得（相对于输入线路速度而言）使所有到达分组无时延地通过它传送，则在输入端口也将出现分组排队，因为到达的分组必须加入输入端口队列中，以等待通过交换结构传送到输出端口。为了举例说明这种排队的重要后果，考虑纵横式交换结构，并假定：①所有链路速度相同；②一个分组能够以一条输入链路接收一个分组所用的相同的时间量，从任意一个输入端口传送到给定的输出端口；③分组按 FCFS 方式，从一指定输入队列移动到其要求的输出队列中。只要其输出端口不同，多个分组可以被并行传送。然而，如果位于两个输入队列前端的两个分组是发往同一输出队列的，则其中的一个分组将被阻塞，且必须在输入队列中等待，因为交换结构一次只能传送一个分组到某指定端口。

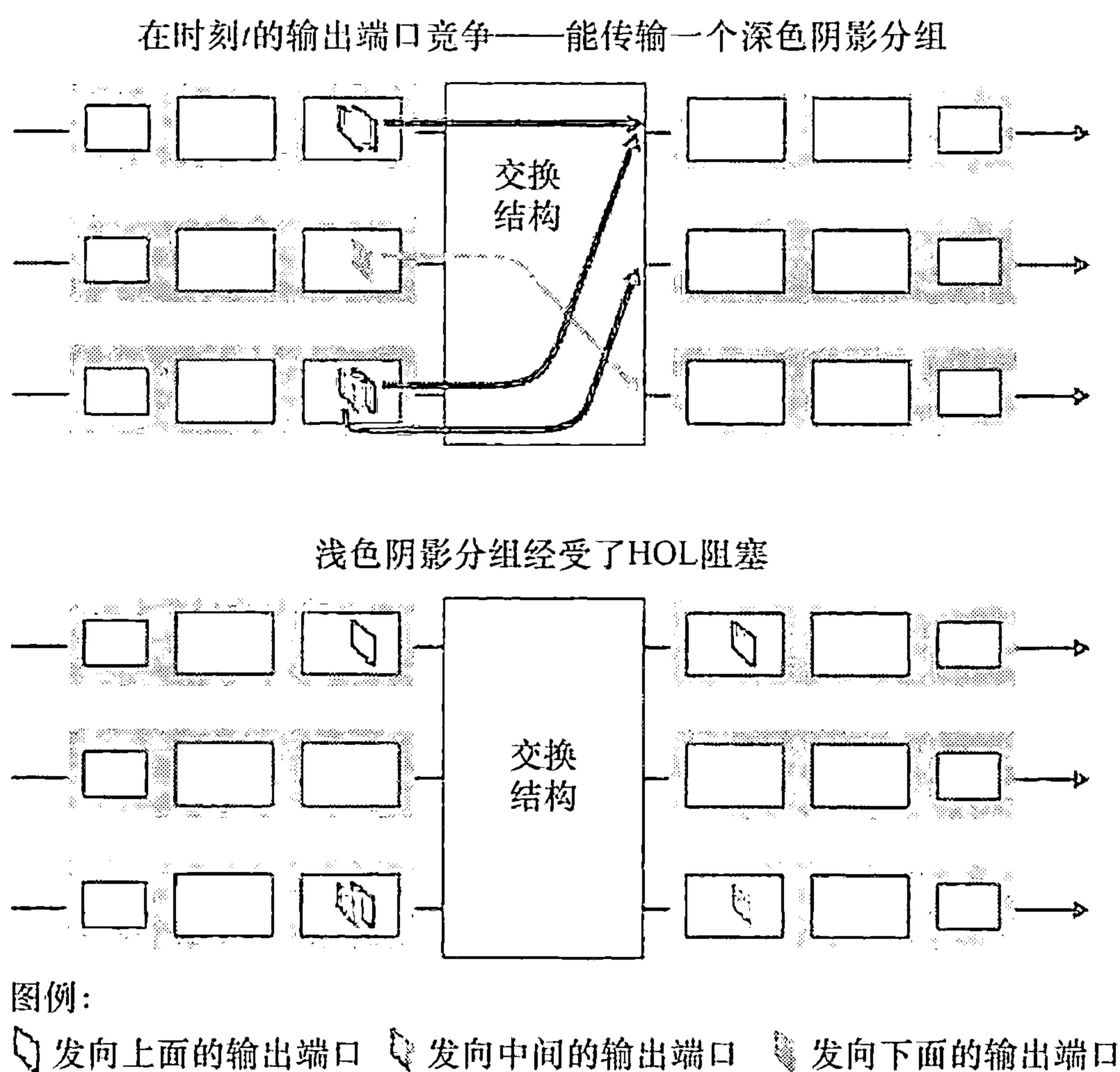


图 4-11 在一个输入排队交换机中的 HOL 阻塞

图 4-11 显示了一个例子，其中在输入队列前端的两个分组（带深色阴影）要发往同一个右上角输出端口。假定该交换结构决定发送左上角队列前端的分组。在这种情况下，

左下角队列中的深色阴影分组必须等待。但不仅该分组要等待，左下角队列中排在该分组后面的浅色阴影分组也要等待，即使右中侧输出端口（浅色阴影分组的目的地）中无竞争。这种现象叫做输入排队交换机中的**线路前部**（Head-Of-the-Line, HOL）阻塞，即在一个输入队列中排队的分组必须等待通过交换结构发送（即使输出端口是空闲的），因为它被位于线路前部的另一个分组所阻塞。[Karol 1987]指出，由于HOL阻塞，只要输入链路上的分组到达速率达到其容量的58%，在某些假设前提下，输入队列长度将无限制地增大（不严格地讲，这等同于说将出现大量的丢包）。[McKeown 1997b]讨论了许多种解决HOL阻塞的方法。

4.3.5 路由选择控制平面

在我们至此所进行的讨论和图4-6中，我们已经隐含地假设路由选择控制平面全部驻留并运行在路由器中的路由选择处理器上。网络范围的路由选择控制平面因此是分布式的，即不同部分（例如路由选择算法）执行在不同的路由器上并且通过彼此发送控制报文进行交互。实际上，今天因特网路由器和我们将在4.6节中学习的路由选择算法正是以这种方式运行的。此外，路由器和交换机厂商将它们的数据平面和软件控制平面绑在一起放入封闭（但可互操作）的平台中，成为一种垂直综合的产品。

近来，一些研究人员[Caesar 2005a; Casado 2009; McKeon 2008]已经着手探索新型路由器控制平面体系结构，其中数据平面连同部分控制平面（例如，链路状态的本地测量/报告，转发表的安装和维护）在路由器中实现，部分控制平面能够在路由器外部实现（例如，在能够执行路由计算的集中式服务器中实现）。一个定义良好的API规定了这两个部分彼此交互和通信的方式。这些研究人员争辩说将软件控制平面与硬件数据平面（具有最少的路由器驻留控制平面）分离，能够以集中式路由选择计算代替分布式路由选择计算，从而简化路由选择，通过允许不同的用户定制的控制平面在快速的硬件数据平面上运行，能够进行网络创新。

4.4 网际协议：因特网中的转发和编址

到目前为止，我们有关网络层编址和转发的讨论并未提及任何特定的计算机网络。在这节中，我们将注意力转向在因特网中是如何完成编址和转发的。我们将看到因特网编址和转发是网际协议（IP）的重要组件。目前有两个版本的IP在使用。我们首先研究广泛部署的IP协议版本4，这通常简称为IPv4[RFC 791]。我们将在本节结尾研究IP版本6[RFC 2460; RFC 4291]，已经提议用它替代IPv4。

在我们尝试研究IP之前，先来考虑构成因特网的网络层的一些组件。如图4-12中所示，因特网的网络层有三个主要组件。第一个组件是IP协议，它是本节的主题。第二个主要组件是路由选择部分，它决定了数据报从源到目的地所流经的路径。我们前面讲过路由选择协议计算出用于在网络中转发分组的转发表。我们将在4.6节中研究因特网的路由选择协议。网络层的最后一个组件是报告数据报中的差错和对某些网络层信息请求进行响应的设施。我们将在4.4.3节中涉及因特网的网络层差错和信息报告的协议，即互联网控制报文协议（ICMP）。